UNC Universidad Nacional de Córdoba

FAMAF Facultad de Matemática, Astronomía, Física y Computación

De Lenguaje Natural a IBDD: Traducción Automática con PLN

por

Anna Paula Aimeri Vicentin

Presentado ante la Facultad de Matemática, Astronomía, Física y Computación como parte de los requerimientos para la obtención del grado de Licenciado/a en Ciencias de la Computación de la

Universidad Nacional de Córdoba

Marzo, 2026

Directora: Laura Brandán Briones

Este trabajo se distribuye bajo una licencia Creative Commons Atribución 4.0 Internacional

Agradecimientos

A Laura, por haber dirigido este trabajo, por confiar en mí, y a Pedro y a Santi, por haber sido parte del tribunal. A los tres, por ser profesores maravillosos.

A Juan, Clau, Lara y Lauti. Estoy orgullosa de ser su familia.

A mis grandes amigas, Nicky, Gali y Ro.

A la Facultad de Matemática, Astronomía, Física y Computación, por mostrarme el mundo platónico de los objetos matemáticos.

A la Universidad Nacional de Córdoba, porque aprender depende de que alguien sostenga un lugar para hacerlo.

Resumen

Este trabajo aborda la automatización de la traducción de escenarios BDD en lenguaje natural hacia representaciones IBDD, etapa inicial del pipeline BDD-a-MBT. Se propone un enfoque híbrido que combina generación con un modelo de lenguaje, validación sintáctica determinística mediante gramática formal IBDD (Lark/Earley) y un ciclo iterativo de corrección guiado por diagnóstico de errores.

La evaluación se realizó con un protocolo reproducible sobre el conjunto de prueba de 90 escenarios (*test*), con partición fija *dev/test* (30/90), cuatro configuraciones lingüísticas (EN–EN, ES–EN, EN–ES, ES–ES), $N = 5$ corridas por configuración y `max-rounds=3`. Los resultados muestran validez sintáctica final alta en todas las configuraciones (84.22 % a 91.11 %), con variabilidad acotada entre corridas. El ciclo de corrección aporta mejoras en todos los casos (ganancias entre 11.78 y 40.22 puntos porcentuales) y recupera una proporción relevante de traducciones inicialmente fallidas (53.03 % a 73.99 %).

El análisis indica que el idioma del prompt se asocia con mayor fuerza a la calidad inicial que el idioma del dataset, y que las diferencias finales se reducen tras la corrección iterativa. También se identifican casos no convergentes al límite de tres rondas, con errores recurrentes en expresiones y listas, lo que delimita líneas concretas de mejora. En conjunto, los resultados sostienen la viabilidad sintáctica del enfoque para producir traducciones IBDD válidas en condiciones experimentales controladas.

Abstract

This thesis addresses the automation of translating natural-language BDD scenarios into IBDD representations, the first stage of the BDD-to-MBT pipeline. We propose a hybrid approach that combines language-model generation, deterministic syntactic validation through a formal IBDD grammar (Lark/Earley), and an iterative correction loop guided by parser-error diagnostics.

The final evaluation follows a reproducible protocol on the 90-scenario test set, using a fixed dev/test split (30/90), four language configurations (EN–EN, ES–EN, EN–ES, ES–ES), $N = 5$ runs per configuration, and `max-rounds=3`. Results show high final syntactic validity across all configurations (84.22 % to 91.11 %), with low run-to-run variability. The correction loop improves outcomes in every configuration (gains from 11.78 to 40.22 percentage points) and recovers a substantial fraction of initially invalid translations (53.03 % to 73.99 %).

The analysis indicates that prompt language is more strongly associated with initial quality than dataset language, while final differences narrow after iterative correction. It also identifies non-convergent cases at the three-round limit, with recurrent failures in expressions and list structures, which defines concrete improvement targets. Overall, the evidence supports the practical syntactic viability of the proposed approach for producing valid IBDD translations under controlled experimental conditions.

Repositorio Público

El código fuente, los prompts, los datasets incluidos y los ejemplos de ejecución utilizados en este trabajo se encuentran disponibles públicamente en:

`https://github.com/annaaimeri/bdd-a-ibdd`

Índice general

1. Introducción **8**
1.1. Motivación y Contexto 8
1.2. Problema 9
1.3. Enfoque propuesto 10
1.4. Contribuciones 11

2. Marco Teórico **12**
2.1. Desarrollo Dirigido por Comportamiento 12
2.1.1. Sintaxis Gherkin 12
2.2. Pruebas Basadas en Modelos 14
2.3. Sistemas de Transición Simbólicos 15
2.4. Lenguaje Intermedio de BDD 16
2.4.1. Información Específica del Dominio 16
2.4.2. Gramática IBDD 17
2.4.3. Pipeline de Transformación 18
2.5. Modelos de Lenguaje Grandes 19
2.6. Trabajos Relacionados 20
2.7. Análisis Sintáctico y Gramáticas Formales 21

3. Metodología **23**
3.1. Selección del Modelo de Lenguaje 23
3.1.1. Metodología de Evaluación 23
3.1.2. Resultados de la Evaluación Comparativa 24
3.1.3. Comparación con la Literatura 26
3.1.4. Modelo Seleccionado 27
3.2. Selección de la Herramienta de Análisis Sintáctico 27
3.2.1. Decisión: Lark con Earley 29
3.3. Diseño del Prompt de Traducción 30
3.3.1. Estructura del Prompt 30
3.3.2. Aprendizaje por Ejemplos 30
3.3.3. Reglas de Traducción 31

3.3.4. Formato de Intercambio 32
3.4. Construcción del Dataset 33
3.4.1. Escenarios Canónicos de Referencia 34
3.4.2. Generación Sintética de Escenarios 34
3.4.3. Dataset en Español 35

4. Implementación **37**
4.1. Arquitectura del Sistema 37
4.2. Definición de la Gramática IBDD 38
4.2.1. Estructura de la Gramática 39
4.2.2. Jerarquía de Expresiones 40
4.2.3. Soporte de Operadores 40
4.2.4. Transformación del Árbol Sintáctico 41
4.3. Módulo de Traducción 41
4.3.1. Abstracción del Proveedor de LLM 41
4.3.2. Structured Outputs de OpenAI 42
4.3.3. Inferencia Dinámica del Esquema 43
4.3.4. Procesamiento de Escenarios 43
4.4. Módulo de Validación Sintáctica 44
4.5. Módulo de Corrección Iterativa 45
4.5.1. Identificación de Casos Fallidos 45
4.5.2. Análisis de Errores 46
4.5.3. Reintento con Contexto de Error 46
4.5.4. Integración de Correcciones 47

5. Evaluación **49**
5.1. Preguntas de Investigación 49
5.2. Experimentos 50
5.2.1. Configuraciones 50
5.2.2. Partición del dataset (desarrollo y prueba) 50
5.2.3. Repeticiones y manejo de la estocasticidad 51
5.2.4. Ciclo de corrección 51
5.2.5. Protocolo de ejecución 52
5.3. Dataset 52
5.4. Métricas 53
5.4.1. Validez sintáctica 53
5.4.2. Ciclo de corrección 54
5.4.3. Estabilidad por caso 55
5.4.4. Métricas operativas 55
5.5. Resultados 55
5.5.1. Casos no convergentes tras tres rondas 57

5.6. Análisis 57
5.6.1. RQ1: validez sintáctica de las traducciones 58
5.6.2. RQ2: efecto del idioma de prompt y dataset 58
5.6.3. RQ3: efectividad del ciclo de corrección iterativo 58
5.6.4. Interpretación de resultados 59
5.6.5. Limitaciones 59
5.6.6. Consideraciones de validez 60

6. Conclusiones y Trabajo Futuro 61
6.1. Conclusiones 61
6.2. Trabajo Futuro 62

A. Prompt de Traducción Utilizado 64
A.1. Contexto 64
A.2. Definición de IBDD 64
A.2.1. Gramática 64
A.2.2. Elementos Clave 65
A.3. Ejemplos de Traducción 65
A.3.1. Ejemplo 1: escenario simple con variable local 65
A.3.2. Ejemplo 2: variables locales con guardas y acciones secuenciales 66
A.3.3. Ejemplo 3: escenario con guarda sin variables locales . 67
A.3.4. Ejemplo 4: múltiples variables de interacción 68
A.4. Formato de Entrada y Salida 68
A.5. Reglas de Traducción 69
A.5.1. Cláusula Given 69
A.5.2. Cláusula When 69
A.5.3. Cláusula Then 69
A.5.4. Convenciones de Nombres 69
A.6. Instrucción Final 70

Capítulo 1

Introducción

1.1. Motivación y Contexto

Uno de los desafíos que enfrenta el desarrollo de software moderno es cerrar la brecha entre la especificación del comportamiento deseado de un sistema y la generación automática de casos de prueba que validen ese comportamiento. El Desarrollo Dirigido por Comportamiento (BDD, *Behavior-Driven Development*) ha demostrado ser efectivo para capturar requisitos mediante escenarios en lenguaje natural estructurado, utilizando la sintaxis Given-When-Then del formato Gherkin. Esta aproximación facilita la colaboración entre técnicos y no técnicos, permitiendo que analistas de negocio, testers y desarrolladores compartan un lenguaje común para describir el comportamiento esperado del sistema.

Por otro lado, las Pruebas Basadas en Modelos (MBT, *Model-Based Testing*) ofrecen un enfoque riguroso y automatizado para generar casos de prueba a partir de modelos formales del sistema, como los Sistemas de Transición Simbólicos (STS, *Symbolic Transition Systems*). Estos modelos formales permiten aplicar algoritmos de generación automática de tests de acuerdo con criterios de cobertura explícitos y detectar comportamientos inesperados.

Sin embargo, existe una brecha entre estos dos mundos. Los escenarios BDD, expresados en lenguaje natural, carecen de la precisión formal necesaria para alimentar directamente las herramientas de MBT. Investigaciones recientes han propuesto el Lenguaje Intermedio de BDD (IBDD) como puente entre ambos paradigmas [36]. IBDD mantiene la estructura Given-When-Then de BDD pero incorpora elementos formales de los STS, incluyendo variables globales y locales, guardas lógicas, gates de interacción y asignaciones explícitas de estado.

Los trabajos de Zameni et al. han demostrado que, una vez que los escenarios están representados en IBDD, es posible transformarlos automáticamente en Sistemas de Transición BDD (BDDTS) y posteriormente en STS, desde donde se pueden generar casos de prueba completos y formalmente válidos [37]. Sin embargo, un eslabón de esta cadena de automatización aún no ha sido abordado: la traducción inicial de escenarios Gherkin escritos por humanos a representaciones IBDD formalmente correctas. La Figura 1.1 ilustra este pipeline y señala la etapa que aborda el presente trabajo.

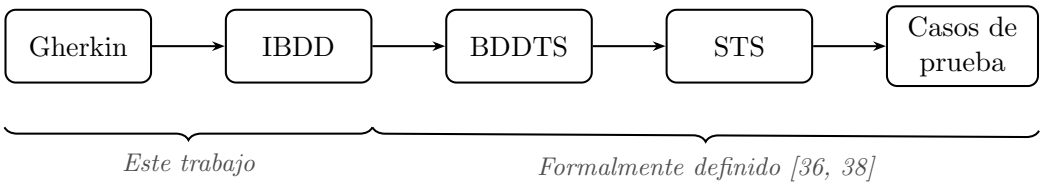

Figura 1.1: Pipeline de transformación BDD-a-MBT y ubicación del presente trabajo

Actualmente, esta traducción es un proceso enteramente manual que requiere experiencia especializada en teoría de sistemas de transición y conocimiento profundo de la gramática IBDD. Un analista debe identificar variables implícitas, inferir gates de interacción apropiados, establecer guardas lógicas correctas y determinar las asignaciones de estado necesarias, información que típicamente no está explícita en el texto natural del escenario BDD. Este cuello de botella limita la escalabilidad industrial de todo el pipeline BDD-a-MBT.

1.2. Problema

El problema central que aborda este trabajo puede formularse así:

¿Cómo automatizar la traducción de escenarios BDD en formato Gherkin al lenguaje intermedio formal IBDD?

Esto requiere preservar razonablemente el comportamiento especificado y generar representaciones formalmente válidas para alimentar herramientas de generación automática de casos de prueba. Este problema presenta múltiples desafíos técnicos. En primer lugar, los escenarios Gherkin contienen imprecisiones inherentes al lenguaje natural que deben resolverse para producir especificaciones formales. A esto se suma que elementos cruciales

de IBDD (variables globales, tipos de datos, funciones del dominio, gates del sistema) no están explícitos en el texto Gherkin y deben inferirse del contexto o del conocimiento del dominio. La correspondencia entre construcciones lingüísticas en Gherkin y elementos formales de IBDD tampoco es trivial; por ejemplo, la frase "el usuario hace clic en agregar al carrito" debe traducirse a un gate de entrada específico con variables de interacción apropiadas. Las traducciones generadas, a su vez, deben ser sintácticamente correctas según la gramática IBDD y semánticamente consistentes con la especificación BDD original. Finalmente, la diversidad de dominios que abarcan los escenarios BDD introduce vocabularios y patrones de interacción específicos que el sistema debe manejar.

1.3. Enfoque propuesto

Este trabajo propone un enfoque híbrido que combina las capacidades de comprensión semántica de Modelos de Lenguaje Grandes (LLMs, *Large Language Models*) con la estructura formal proporcionada por la gramática IBDD y técnicas de análisis sintáctico determinístico.

La arquitectura del sistema se organiza en torno a dos componentes principales. El primero es un módulo de traducción basado en LLM que utiliza modelos de lenguaje para interpretar escenarios Gherkin y generar candidatos de traducción IBDD, aprovechando la capacidad de los LLMs para capturar patrones semánticos complejos y realizar inferencias sobre información implícita. El segundo componente es un módulo de validación sintáctica para verificar las traducciones generadas contra la gramática formal IBDD y garantizar corrección estructural.

El flujo del sistema combina una etapa de traducción de escenarios BDD en formato Gherkin asistida por LLM con una etapa de validación sintáctica mediante el analizador Lark. Cuando una traducción no supera la validación, se activa un ciclo de corrección iterativa orientado a mejorar la conformidad con la gramática IBDD.

La elección de LLMs se fundamenta en su demostrada capacidad para tareas de traducción entre lenguajes formales y naturales, como se evidencia en trabajos recientes sobre generación de código desde especificaciones [14] y traducción de requisitos a modelos formales [29]. Sin embargo, los LLMs por sí solos no garantizan corrección sintáctica ni consistencia formal, de ahí la necesidad del componente de validación.

1.4. Contribuciones

Este trabajo contribuye con el diseño e implementación de un pipeline automatizado para la traducción BDD→IBDD que integra generación con LLM, validación sintáctica determinística y corrección iterativa de traducciones fallidas. También formaliza una estrategia de validación operativa basada en la gramática IBDD y parser Earley (Lark), con reportes estructurados de error reutilizables en el ciclo de reintento.

En el plano experimental, se construye y documenta un protocolo reproducible con partición fija del conjunto de datos en desarrollo y prueba, múltiples corridas por configuración y métricas agregadas para controlar la estocasticidad del modelo. Sobre esa base, se realiza una evaluación cuantitativa en cuatro configuraciones lingüísticas de entrada/salida sobre el conjunto de prueba, permitiendo analizar validez inicial y final, ganancia por corrección y desempeño del proceso de corrección. Finalmente, el trabajo delimita el alcance actual del enfoque, analizando calidad sintáctica y señalando la fidelidad semántica como eje de extensión futura.

Capítulo 2

Marco Teórico

2.1. Desarrollo Dirigido por Comportamiento

El Desarrollo Dirigido por Comportamiento (BDD, *Behavior-Driven Development*) es una práctica de desarrollo de software que surge como evolución del Desarrollo Dirigido por Pruebas (TDD, *Test-Driven Development*). Fue propuesto por Dan North en 2006 [20] como respuesta a las dificultades que encontraban los equipos al adoptar TDD: incertidumbre sobre qué probar, cómo nombrar las pruebas y cómo entender por qué una prueba falla.

La innovación central de BDD consiste en desplazar el foco desde las pruebas hacia el comportamiento esperado del sistema. En lugar de escribir pruebas unitarias que verifican implementaciones específicas, BDD propone describir el comportamiento del sistema desde la perspectiva del usuario mediante escenarios concretos. Este cambio de vocabulario (de "prueba" a "comportamiento") tiene implicaciones profundas: facilita la comunicación entre desarrolladores, analistas de negocio y otros *stakeholders*, y produce especificaciones que sirven simultáneamente como documentación y como base para pruebas automatizadas.

Solís y Wang [30] identifican como características fundamentales de BDD la ubicuidad del lenguaje compartido entre todos los participantes del proyecto, la naturaleza iterativa del proceso de especificación, y el uso de ejemplos concretos como mecanismo primario de comunicación entre el dominio del negocio y el dominio técnico.

2.1.1. Sintaxis Gherkin

El vehículo principal para expresar escenarios BDD es Gherkin, un lenguaje de dominio específico diseñado para ser legible tanto por humanos como por herramientas de automatización [4]. Un escenario Gherkin describe una

situación concreta del comportamiento del sistema mediante tres cláusulas estructuradas:

- **Given** (*Dado*): establece las precondiciones o el estado inicial del sistema antes de que ocurra la acción bajo prueba.
- **When** (*Cuando*): describe la acción o evento que desencadena el comportamiento que se desea especificar.
- **Then** (*Entonces*): define el resultado esperado o la postcondición que debe verificarse tras la ejecución de la acción.

A modo de ejemplo, el siguiente escenario especifica el comportamiento de un sistema de comercio electrónico al agregar un producto al carrito de compras:

Listing 2.1 Ejemplo de un escenario BDD en sintaxis Gherkin

```
Scenario: Add product to shopping cart
  Given the user is on the product page for "Laptop"
  And the product is in stock
  When the user clicks the "Add to Cart" button
  Then the cart should contain 1 item
  And the total price should be updated
```

Los escenarios Gherkin se agrupan en archivos denominados *features*, donde cada *feature* describe una funcionalidad del sistema e incluye uno o más escenarios que ilustran su comportamiento esperado bajo distintas condiciones. Las palabras clave **And** y **But** permiten encadenar múltiples condiciones dentro de una misma cláusula, mejorando la legibilidad sin alterar la semántica de la especificación.

Esta estructura, si bien es lo suficientemente formal para ser procesada por herramientas de automatización como Cucumber[1], mantiene un nivel de abstracción cercano al lenguaje natural. Es precisamente esta característica la que convierte a Gherkin en un formato accesible para *stakeholders* no técnicos, pero también la que introduce desafíos de ambigüedad e imprecisión inherentes al uso del lenguaje natural. Estos desafíos se vuelven críticos cuando se busca integrar los escenarios BDD con técnicas de verificación formal, como las Pruebas Basadas en Modelos.

[1]Cucumber es un *framework* de código abierto que ejecuta especificaciones Gherkin como pruebas automatizadas, vinculando cada paso del escenario con código de prueba mediante expresiones regulares [3].

2.2. Pruebas Basadas en Modelos

Las Pruebas Basadas en Modelos (MBT, *Model-Based Testing*) constituyen un enfoque de verificación de software en el que los casos de prueba se derivan de manera rigurosa a partir de un modelo formal que describe el comportamiento esperado del sistema bajo prueba [32]. A diferencia de las técnicas de prueba manuales, donde un ingeniero diseña casos de prueba individualmente, MBT automatiza este proceso: dado un modelo del sistema y un criterio de cobertura, un algoritmo de generación produce un conjunto de casos de prueba que satisface dicho criterio.

Utting et al. [32] proponen una taxonomía que clasifica los enfoques de MBT según cinco dimensiones: la notación del modelo (máquinas de estados, autómatas, modelos de flujo de datos), el criterio de selección de pruebas (cobertura estructural, cobertura de requisitos, selección aleatoria), la tecnología de generación (model checking, ejecución simbólica, búsqueda), la ejecución de las pruebas (*online* versus *offline*), y el nivel de automatización del proceso completo.

En el contexto de sistemas reactivos, es decir, sistemas que interactúan con su entorno mediante estímulos y respuestas, Tretmans [31] formaliza la relación de conformidad *ioco* (*input-output conformance*) como criterio para determinar si una implementación se comporta de acuerdo con su especificación. Bajo esta teoría, el modelo del sistema se expresa como un Sistema de Transición Etiquetado (*Labelled Transition System*, LTS), y los casos de prueba se generan como secuencias de estímulos y observaciones que verifican la conformidad de la implementación con respecto al modelo.

El atractivo fundamental de MBT reside en la posibilidad de generar automáticamente grandes conjuntos de casos de prueba con garantías formales de cobertura, lo cual resulta particularmente valioso en dominios donde la exhaustividad de las pruebas es crítica. Sin embargo, la aplicación de MBT requiere la existencia de un modelo formal del sistema, cuya construcción constituye en sí misma un desafío significativo. En la tesis doctoral de Brandán Briones [2] se desarrollan teorías para MBT que abordan aspectos de tiempo real y cobertura en este tipo de sistemas. Entre los formalismos utilizados como base para MBT se encuentran los Sistemas de Transición Simbólicos (STS), fundamentales para representar de manera simbólica estados, transiciones y restricciones sobre datos.

2.3. Sistemas de Transición Simbólicos

Los Sistemas de Transición Simbólicos (STS, *Symbolic Transition Systems*) constituyen una extensión de los Sistemas de Transición Etiquetados (LTS) que incorpora una representación explícita de datos y flujo de control dependiente de datos [10]. Mientras que en un LTS los valores de datos se representan de manera concreta, lo cual conduce a una explosión del espacio de estados cuando los dominios de datos son grandes o infinitos, los STS operan a nivel simbólico, utilizando variables y fórmulas de lógica de primer orden para expresar condiciones sobre los datos.

Formalmente, un STS se define como una tupla $\langle L, l_0, V, \iota, I, \Lambda, \rightarrow \rangle$, donde [9]:

- L es un conjunto finito de *localidades* (estados de control).
- $l_0 \in L$ es la localidad inicial.
- V es un conjunto de *variables de localidad* que representan el estado del sistema.
- ι es una valuación inicial que asigna valores a las variables en V.
- I es un conjunto de *variables de interacción* utilizadas para el intercambio de datos con el entorno.
- Λ es un conjunto de *gates* (etiquetas de interacción) que representan los canales de comunicación.
- $\rightarrow \subseteq L \times \Lambda \times \Phi \times F \times L$ es la relación de *switches* (transiciones), donde Φ denota guardas lógicas y F funciones de actualización de variables.

Cada switch representa una transición entre dos localidades, condicionada por una guarda (una fórmula lógica sobre las variables de localidad y de interacción) y acompañada por una función de actualización que modifica el estado. Los gates se clasifican en gates de entrada (que reciben estímulos del entorno) y gates de salida (que emiten respuestas), reflejando la naturaleza reactiva de los sistemas modelados.

La ventaja de esta representación simbólica es doble. En primer lugar, evita la enumeración explícita de valores de datos, lo cual permite modelar sistemas con dominios infinitos sin incurrir en explosión de estados. En segundo lugar, preserva la información semántica de las restricciones sobre los datos, que puede ser explotada durante la generación de casos de prueba. Frantzen et al. [9] definen la relación de conformidad simbólica *sioco*, que extiende la relación *ioco* de Tretmans al nivel simbólico, proporcionando una base formal para la generación de pruebas directamente desde especificaciones STS.

2.4. Lenguaje Intermedio de BDD

El Lenguaje Intermedio de BDD (IBDD, *Intermediate Behavior-Driven Development*) fue propuesto por Zameni et al. [36] como puente formal entre los escenarios textuales BDD y los modelos formales requeridos por las técnicas de MBT. IBDD conserva la estructura Given-When-Then de BDD pero incorpora elementos de los Sistemas de Transición Simbólicos, añadiendo la precisión formal necesaria para habilitar transformaciones automáticas hacia modelos de prueba. En particular, adopta directamente la terminología y estructura de los STS (variables, gates, guardas y asignaciones), por lo que los conceptos introducidos en la Sección 2.3 resultan centrales para su interpretación.

La motivación de IBDD radica en que los escenarios BDD textuales presentan dos limitaciones fundamentales para su uso en MBT: ambigüedad inherente al lenguaje natural e incompletitud de la información necesaria para construir modelos formales. Elementos como las variables de estado del sistema, los tipos de datos, las guardas lógicas y los canales de interacción no están explícitos en el texto Gherkin y deben ser inferidos por un experto del dominio. IBDD hace explícita esta información mediante una gramática híbrida que combina notación EBNF con estructuras matemáticas de lógica de primer orden.

2.4.1. Información Específica del Dominio

IBDD introduce el concepto de Información Específica del Dominio (DSI, *Domain-Specific Information*), definida como una tupla $(\mathcal{S}, \mathcal{F}, \mathcal{V}, \mathcal{G})$, donde [36]:

- $\mathcal{S}$ es el conjunto de *sorts* (tipos de datos), que incluye tipos primitivos (Bool, Int), tipos compuestos, enumeraciones y tipos lista.
- $\mathcal{F}$ es el conjunto de funciones definidas sobre los tipos de datos del conjunto $\mathcal{S}$.
- $\mathcal{V}$ es el conjunto de variables tipadas, particionado en tres subconjuntos disjuntos: variables *globales* (representan el estado persistente del sistema), variables *locales* (con alcance limitado al escenario) y variables de *interacción* (utilizadas para el intercambio de datos en una transición).
- $\mathcal{G}$ es el conjunto de *gates*, particionado en gates de entrada $\mathcal{G}_i$ (iniciados por el entorno, prefijados con `?`) y gates de salida $\mathcal{G}_o$ (iniciados por el sistema, prefijados con `!`).

2.4.2. Gramática IBDD

En esta sección se presenta la estructura sintáctica general de IBDD tal como se describe en la literatura de referencia. En este trabajo, esa base se operacionaliza mediante una gramática en EBNF compatible con Lark, utilizada por el parser del sistema.

La gramática IBDD estructura cada escenario en tres cláusulas:

- **Given**: declara variables locales e introduce una guarda de entrada (*input guard*), que es una conjunción de predicados sobre variables globales y locales representando la precondición del escenario.
- **When**: consiste en uno o más *switches*, cada uno compuesto por una interacción (un gate con sus variables de interacción), una condición (un predicado booleano) y una asignación (una función de actualización de variables).
- **Then**: consiste en cero o más switches de salida seguidos de una guarda de salida (*output guard*) que define la postcondición del escenario.

A modo de ejemplo, el siguiente término IBDD representa un escenario del dominio de impresora industrial donde un trabajo de impresión se mueve a la lista de trabajos completados [36]:

Listing 2.2 Ejemplo de un término IBDD para un escenario de impresora industrial

```
GIVEN CJ [is_in_list(CJ, SJL) ∧ CJ.type = Production]
WHEN !printstart.cj
       cj.id = CJ.id ∧ cj.state = Printing
       CJ.state := cj.state
     !printcomplete.cj
       cj.id = CJ.id ∧ cj.state = Completed
       CJ.state := cj.state, PJL := add(CJ, PJL)
THEN [is_in_list(CJ, PJL)]
```

En este ejemplo, `CJ` es una variable local que representa un trabajo de control, `SJL` y `PJL` son variables globales que representan las listas de trabajos programados e impresos, y `!printstart` y `!printcomplete` son gates de salida del sistema. Las condiciones verifican la identidad y el estado del trabajo, mientras que las asignaciones actualizan el estado de las variables.

2.4.3. Pipeline de Transformación

IBDD se sitúa en el centro de un pipeline de transformación que conecta los escenarios BDD con la generación automática de casos de prueba. Este pipeline fue planteado progresivamente en la literatura, en particular por los trabajos de Zameni et al., y consta de las siguientes etapas:

1. **BDD → IBDD**: traducción de escenarios Gherkin al formalismo IBDD, definida por Zameni et al. [36] en 2024 junto con la especificación del lenguaje. Esta etapa ha sido realizada de forma manual, ya que requiere experiencia especializada en la sintaxis IBDD y en teoría de sistemas de transición.

2. **IBDD → BDDTS**: transformación formal de términos IBDD a Sistemas de Transición BDD (*BDD Transition Systems*), una variante de STS que particiona las variables de estado en locales y globales, y añade una localidad objetivo junto con guardas de entrada y salida. Esta transformación fue formalizada por Zameni et al. [36] en 2024, aunque una versión preliminar, sin la etapa intermedia de IBDD, había sido presentada en 2023 [37].

3. **BDDTS a STS**: transformación de los BDDTS a Sistemas de Transición Simbólicos. Zameni et al. [36] esbozaron tres enfoques posibles para esta transformación en 2024, y posteriormente formalizaron uno de ellos en 2025 [38].

4. **STS → Tests**: aplicación de algoritmos de generación automática de casos de prueba sobre los STS resultantes. Esta etapa se apoya en algoritmos preexistentes desarrollados por Frantzen et al. [9] y van den Bos y Tretmans [1].

Zameni et al. [38] extienden este pipeline en 2025 con un operador formal de composición secuencial de BDDTS, que permite construir modelos complejos a partir de escenarios individuales. Los autores demuestran la corrección del operador mediante pruebas de solidez y completitud, y reportan su aplicación a un caso de estudio industrial de sistemas de iluminación aeroportuaria. Hasta el momento, la primera etapa de este pipeline, la traducción de escenarios Gherkin a IBDD, no ha sido abordada de forma automática en la literatura. La automatización de esta etapa requiere un mecanismo capaz de interpretar texto en lenguaje natural y generar representaciones formales, capacidad que ofrecen los Modelos de Lenguaje Grandes.

2.5. Modelos de Lenguaje Grandes

Los Modelos de Lenguaje Grandes (LLMs, *Large Language Models*) son redes neuronales con miles de millones de parámetros, entrenadas sobre corpus masivos de texto para predecir la siguiente secuencia de tokens en un contexto dado. Su arquitectura se fundamenta en el *Transformer* [33], que introdujo el mecanismo de *self-attention* como alternativa a las redes recurrentes, permitiendo capturar dependencias a larga distancia en secuencias de texto de forma paralelizable y eficiente.

A partir de esta arquitectura base, modelos sucesivos han demostrado capacidades emergentes que trascienden la generación de texto: comprensión semántica profunda, razonamiento sobre estructuras formales, y generación de código en múltiples lenguajes de programación. Entre las familias de modelos más representativas se encuentran la familia GPT de OpenAI [21], Claude de Anthropic y Llama de Meta, que difieren en sus orígenes (propietario de uso general, orientado a razonamiento y de código abierto, respectivamente), pero comparten la capacidad de procesar y generar texto estructurado con alto grado de coherencia. La selección del modelo más adecuado para una tarea específica requiere una evaluación comparativa que considere factores como la corrección de las salidas, la consistencia entre ejecuciones y la capacidad de adherirse a formatos formales.

Un parámetro central en la generación de texto con LLMs es la *temperatura* (T), que controla la aleatoriedad de la selección de tokens. Durante la generación, el modelo asigna una probabilidad a cada token candidato. Con $T = 0$ el modelo prioriza tokens de alta probabilidad y el comportamiento es teóricamente más determinístico, aunque en la práctica pueden aparecer candidatos con probabilidades similares y la opción más probable no siempre coincide con la más correcta. Valores mayores de T aplanan la distribución de probabilidades, aumentando la diversidad de las salidas a costa de una mayor variabilidad entre ejecuciones [12]. En aplicaciones prácticas suele utilizarse T en el rango $[0, 1]$, y el efecto preciso de variar este parámetro depende de la tarea, el modelo y el protocolo experimental [27]. Por ello, este parámetro tiene implicaciones directas para tareas que requieren corrección formal, pero su efecto sobre la validez sintáctica y la calidad final debe evaluarse empíricamente para cada tarea.

La aplicación de LLMs en ingeniería de software ha experimentado un crecimiento notable. Jiang et al. [14] presentan un relevamiento exhaustivo que identifica las principales tareas en las que los LLMs han demostrado efectividad: generación de código a partir de descripciones en lenguaje natural, completado automático de código, detección y corrección de errores, generación de pruebas, y traducción entre lenguajes de programación. Herramientas como

GitHub Copilot ilustran la adopción industrial de estas capacidades.

En el contexto específico de BDD, Karpurapu et al. [15] evalúan el uso de LLMs para la formulación automática de pruebas de aceptación. Sus hallazgos indican que la técnica de *few-shot prompting*, en la que se incluyen ejemplos resueltos de la tarea como parte de la entrada al modelo, mejora significativamente la precisión de las traducciones generadas en comparación con estrategias de *zero-shot prompting*, en las que el modelo recibe únicamente la instrucción sin ejemplos de referencia.

A pesar de sus capacidades, los LLMs presentan limitaciones relevantes para tareas que requieren corrección formal. Los modelos pueden generar salidas sintácticamente incorrectas, introducir *hallucinations* (contenido plausible pero factualmente incorrecto), y exhibir inconsistencias entre ejecuciones sucesivas. Estas limitaciones son particularmente críticas cuando el objetivo es producir representaciones formales, donde un error sintáctico puede invalidar completamente la salida generada. Estas limitaciones motivan la necesidad de complementar los LLMs con mecanismos de verificación formal, aspecto que se abordará tras revisar los trabajos relacionados.

2.6. Trabajos Relacionados

En su trabajo inicial [37], Zameni et al. presentaron la idea de transformar manualmente escenarios BDD en modelos formales sobre los cuales aplicar técnicas de MBT, demostrando la viabilidad del enfoque pero señalando la dificultad de automatizar la transformación debido a la gran discrepancia entre los escenarios textuales y los modelos formales. Posteriormente, Zameni et al. [36] definieron el lenguaje intermedio IBDD y formalizaron la transformación de IBDD a BDDTS y de BDDTS a STS, estableciendo así los eslabones formales de la cadena. Más recientemente [38], extendieron el framework con mecanismos de composición secuencial de BDDTS que permiten construir modelos complejos a partir de escenarios individuales. Sin embargo, la traducción inicial de escenarios Gherkin a IBDD no ha sido explorada de forma automática hasta el momento.

En una línea distinta, la aplicación de técnicas de procesamiento de lenguaje natural a escenarios BDD tiene antecedentes previos a la era de los LLMs. Soeken et al. [29] propusieron un flujo asistido para BDD donde el sistema utiliza NLP para extraer semi-automáticamente la estructura del código de implementación y los casos de prueba a partir de escenarios textuales, estableciendo un diálogo interactivo con el usuario. Este enfoque opera a nivel de código fuente y no genera representaciones formales intermedias.

Fischbach et al. [8] desarrollaron SPECMATE, una herramienta que

automatiza la creación de casos de prueba a partir de criterios de aceptación expresados en lenguaje natural. SPECMATE utiliza técnicas de NLP para extraer relaciones causales de los criterios de aceptación y generar árboles de causa-efecto, a partir de los cuales se derivan automáticamente los casos de prueba. SPECMATE no genera representaciones formales intermedias ni se integra con frameworks de MBT basados en STS.

Más recientemente, Karpurapu et al. [15] realizaron una evaluación exhaustiva del uso de LLMs para la formulación automática de pruebas de aceptación en BDD. Sus experimentos abarcan múltiples modelos y estrategias de prompting, y sus resultados demuestran que la técnica de *few-shot prompting*, en la que se proveen ejemplos de la tarea al modelo, mejora significativamente la calidad de las salidas en comparación con *zero-shot prompting*.

Por otra parte, los trabajos de Karpurapu et al. [15] evidencian el potencial de estos modelos para tareas de generación estructurada en el contexto de BDD, aunque su enfoque se concentra en la dirección inversa (historias de usuario a Gherkin) y no contempla la integración con mecanismos de validación sintáctica determinísticos basados en gramáticas formales.

En cuanto al pipeline formal, Zameni et al. [36, 37, 38] han formalizado las transformaciones desde IBDD hasta la generación de casos de prueba; sin embargo, hasta donde sabemos, la primera etapa, la traducción de escenarios Gherkin a IBDD, no parece haber sido abordada de forma automática en los trabajos existentes.

El presente trabajo se sitúa en la convergencia de estas líneas de investigación, proponiendo un enfoque que articula la capacidad de comprensión semántica de los LLMs con la validación sintáctica determinística mediante gramáticas formales, aplicado específicamente a la traducción de escenarios Gherkin al formalismo IBDD. El componente de validación formal de este enfoque se sustenta en técnicas de análisis sintáctico basadas en gramáticas formales.

2.7. Análisis Sintáctico y Gramáticas Formales

Dado que el enfoque propuesto incorpora validación sintáctica automática de las traducciones generadas, resulta necesario introducir los fundamentos formales que sostienen ese componente. En particular, interesa considerar la notación EBNF, el algoritmo de Earley y la biblioteca Lark, ya que constituyen la base del validador sintáctico utilizado en este trabajo.

La gramática IBDD se especifica utilizando la notación EBNF (*Extended*

Backus-Naur Form), propuesta por Niklaus Wirth [35] como extensión de la notación BNF (*Backus-Naur Form*) introducida por John Backus en 1959 para describir la sintaxis de ALGOL 60. Wirth observó que la diversidad innecesaria de notaciones sintácticas en la literatura dificultaba la comunicación, y propuso EBNF como notación unificada que incorpora operadores de expresiones regulares: corchetes `[ ]` para secuencias opcionales, llaves `{ }` para repeticiones, y paréntesis `( )` para agrupamiento. Estas extensiones permiten expresar gramáticas de manera más compacta, evitando el uso intensivo de recursión para representar repeticiones simples.

En este marco, el algoritmo de Earley [6] resulta especialmente relevante porque es un algoritmo de análisis descendente basado en programación dinámica que opera sobre gramáticas libres de contexto arbitrarias. A diferencia de los analizadores LR o LALR, que requieren gramáticas restringidas a subclases específicas, el algoritmo de Earley acepta cualquier gramática libre de contexto sin necesidad de transformaciones previas.

El algoritmo procesa la cadena de entrada de izquierda a derecha, manteniendo un conjunto de *items* (reglas de producción parcialmente reconocidas) organizados en columnas correspondientes a cada posición de la entrada. En cada paso, tres operaciones fundamentales avanzan el reconocimiento: *prediction* (predice qué reglas pueden aplicarse), *scanning* (consume un terminal de la entrada) y *completion* (propaga el reconocimiento de un no-terminal completado). Su complejidad temporal es $O(n^3)$ en el caso general, $O(n^2)$ para gramáticas no ambiguas, y $O(n)$ para gramáticas LR(k), donde n es la longitud de la entrada.

Una ventaja significativa del algoritmo de Earley es su capacidad para operar con gramáticas que no se ajustan a las restricciones de los analizadores LALR convencionales, como aquellas que combinan elementos EBNF con estructuras matemáticas de lógica de primer orden [6, 11]. Además, su flexibilidad permite modificar y extender gramáticas sin necesidad de reestructurarlas para satisfacer requisitos de analizadores más restrictivos [11].

Sobre esta base, Lark [18] constituye una biblioteca de análisis sintáctico para Python que soporta múltiples algoritmos, incluyendo Earley y LALR(1)[2]. Lark permite definir gramáticas en una variante de EBNF y genera automáticamente el analizador correspondiente, proporcionando árboles de derivación tipados y mensajes de error detallados que facilitan el diagnóstico de fallos sintácticos. Estas características la convierten en una alternativa adecuada para implementar la validación sintáctica del lenguaje IBDD.

[2]En LALR(1), el "1" indica que el analizador utiliza un token de anticipación (*lookahead*) para resolver decisiones de análisis sintáctico.

Capítulo 3

Metodología

La metodología combina decisiones de ingeniería y criterios experimentales. En el plano de ingeniería, se prioriza la modularidad (separación entre cliente LLM, traductor, validador y corrector) para facilitar reproducibilidad y evolución del sistema. En el plano experimental, se establece un protocolo con configuraciones lingüísticas controladas, partición fija del dataset y repetición de corridas para estimar desempeño medio y variabilidad.

3.1. Selección del Modelo de Lenguaje

La tarea central de este trabajo, traducir escenarios BDD en lenguaje natural a representaciones formales IBDD, requiere un modelo de lenguaje con capacidades de comprensión semántica, generación estructurada y consistencia sintáctica. Esta sección describe el proceso de evaluación comparativa que condujo a la selección del modelo utilizado.

3.1.1. Metodología de Evaluación

Se evaluaron tres modelos ampliamente utilizados: GPT-4o de OpenAI, Claude Sonnet 4 de Anthropic y Llama 3.3 70B de Meta. La selección de estos modelos responde a la diversidad de sus orígenes (propietario de uso general, propietario orientado a razonamiento, y de código abierto) y a su relevancia práctica al momento de la evaluación.

La evaluación se realizó sobre un escenario de *e-commerce* que involucra interacciones típicas de sistemas web: gestión de productos, carritos de compra, inventario y sesiones de usuario. Este escenario presenta desafíos característicos de la traducción BDD→IBDD, ya que requiere identificar variables globales (inventario, carrito) y locales (producto, cantidad), inferir gates de interacción

apropiados (`?click`, `?input`, `!add`, `!update`), establecer guardas lógicas para validaciones y definir asignaciones de estado correctas.

Escenario de Prueba

El escenario BDD utilizado para la evaluación fue el siguiente:

```
Scenario: A product is added to shopping cart when user selects it
  Given a product is available in inventory
  And the user is logged in
  When the user clicks add to cart button
  And the user specifies quantity
  Then the product is added to the shopping cart
  And the inventory is updated
```

Diseño del Prompt

Para la evaluación se diseñó un prompt preliminar que incorpora la definición completa de la gramática IBDD en notación EBNF, una explicación de los elementos clave del lenguaje (variables locales y globales, gates, guards y assignments), un ejemplo canónico de referencia del dominio de impresora industrial extraído de los trabajos de Zameni et al. [36], e instrucciones para generar tres opciones de traducción con enfoques diferenciados. Los criterios de evaluación abarcaron la granularidad de las interacciones, el modelado de variables, el nivel de detalle y la estructuración de estados. Este prompt se aplicó de manera idéntica a los tres modelos para garantizar la comparabilidad de los resultados. La solicitud de tres opciones por modelo permitió evaluar tanto la capacidad de traducción como la diversidad de los enfoques propuestos.

3.1.2. Resultados de la Evaluación Comparativa

Se obtuvieron nueve traducciones en total (tres opciones por cada modelo). La Tabla 3.1 resume los resultados según los criterios evaluados.

GPT-4o

GPT-4o demostró el desempeño más consistente entre los tres modelos. Las tres opciones generadas respetan la gramática IBDD sin desviaciones sintácticas, y cada una explora un paradigma técnico distinto: atómico/granular, compuesto y concurrente con reservas. El siguiente fragmento ilustra una de las traducciones generadas, correspondiente al enfoque compuesto:

Tabla 3.1: Comparación de modelos para la traducción BDD→IBDD

Criterio	**GPT-4o**	**Claude S. 4**	**Llama 3.3**
Corrección sintáctica	Alta	Media	Baja
Consistencia	Alta	Media	Baja
Diversidad de enfoques	Alta	Alta	Baja
Preservación semántica	Alta	Alta	Media
Calidad de justificaciones	Alta	Media	Baja

```
GIVEN PROD, USER [is_available(PROD, INV) ∧ is_logged_in(USER)]
WHEN
    ?addToCart.prod_qty, payload
        payload.product = PROD ∧ payload.qty > 0
        (PROD, QTY) := (payload.product, payload.qty)
THEN
    !persist.cart_and_inventory, result
        result.cart_after = addItem(S_CART, PROD, QTY) ∧
        result.inv_after = INV[PROD] - QTY
        CART := result.cart_after;
        INV := updateInventory(INV, PROD, QTY)
    [is_in_cart(PROD, CART) ∧ INV[PROD] = INV_before(PROD) - QTY]
```

Esta traducción modela la interacción del usuario como un gate de entrada (`?addToCart`), define guardas que verifican la disponibilidad del producto y la validez de la cantidad, y establece asignaciones que reflejan las modificaciones al carrito y al inventario. El propio modelo identifica correctamente los compromisos de diseño entre las opciones, señalando por ejemplo que la tercera opción puede resultar sobredimensionada. En efecto, algunas opciones tienden a introducir niveles de complejidad que, si bien son técnicamente válidos, exceden lo requerido por el escenario, y las explicaciones que acompañan cada traducción resultan a veces verbosas.

Claude Sonnet 4

Claude exhibió creatividad conceptual en sus traducciones, con una progresión deliberada de simplicidad a complejidad entre las tres opciones, una documentación detallada de las variables utilizadas y guardas lógicas bien estructuradas. Sin embargo, la consistencia sintáctica fue inferior a la de GPT-4o. En particular, una de las opciones introduce notaciones como `BTN_STATE := clicked` que no se ajustan a las convenciones de la gramática IBDD de referencia:

```
GIVEN P, U, Q [P.available = true ∧ U.logged_in = true]
WHEN ?click.add_to_cart_btn
     U.logged_in = true ∧ P.available = true
     BTN_STATE := clicked
```

Si bien la traducción es conceptualmente correcta, estas desviaciones sintácticas requerirían corrección posterior para ser procesadas por el analizador IBDD.

Llama 3.3 70B

Llama 3.3 presentó las mayores limitaciones. Si bien las traducciones son breves y no contienen errores conceptuales graves, las tres opciones presentan errores sintácticos consistentes, ninguna modela la actualización de inventario (perdiendo información del escenario original), las justificaciones de diseño carecen de sustento técnico, y se confunde la dirección de los gates de interacción. El siguiente fragmento ilustra estos problemas:

```
GIVEN prod [true], user logged_in
WHEN !select_prod, ?spec_quantity
     true
     prod := p
THEN !add_to_cart.sc
     sc.product = p ∧ sc.quantity = q
     SC := add(p, sc), Q := spec_quantity
     [is_in_SC(p, sc) ∧ sc.product = p]
```

En esta traducción, el gate `!select_prod` (gate de salida) se utiliza para modelar una acción del usuario, cuando la semántica de IBDD establece que las acciones del usuario corresponden a gates de entrada (`?`). La combinación de un gate de salida con uno de entrada en la cláusula `WHEN` viola la estructura esperada del lenguaje.

3.1.3. Comparación con la Literatura

Los resultados obtenidos son consistentes con hallazgos de la literatura. Karpurapu et al. [15] evaluaron varios LLMs (GPT-3.5, GPT-4, Llama-2-13B, PaLM-2) para la generación automática de tests de aceptación BDD a partir de historias de usuario. En ese estudio, los modelos GPT reportan menor cantidad de errores sintácticos y mayor adherencia a reglas de Gherkin que Llama-2-13B, y la estrategia *few-shot* muestra mayor precisión que *zero-shot*.

Aunque el trabajo de Karpurapu et al. aborda la dirección inversa a la de este estudio (historias de usuario → Gherkin, frente a Gherkin → IBDD), los patrones observados son comparables: en ese estudio los modelos GPT muestran mejor consistencia sintáctica que Llama, y la provisión de ejemplos mejora los resultados. Esta convergencia aporta respaldo adicional para la selección realizada.

3.1.4. Modelo Seleccionado

Sobre la base de los resultados experimentales y su consistencia con la literatura, se seleccionó el modelo GPT-4o de OpenAI como motor de lenguaje del sistema de traducción. En la comparación realizada, este modelo mostró corrección sintáctica en las traducciones generadas para el escenario evaluado, buena uniformidad entre ejecuciones y capacidad de producir variantes de traducción técnicamente diferenciadas.

No obstante, una decisión de diseño fundamental del sistema es evitar el acoplamiento a un modelo específico. Por ello, la arquitectura abstrae la comunicación con el LLM mediante una capa con proveedor y modelo parametrizados como variables de configuración. En la implementación actual, esta abstracción permite intercambiar modelos dentro de los proveedores soportados y mantener estable el resto del pipeline.

Esta decisión responde a tres consideraciones principales. La primera es la soberanía de datos: en contextos institucionales y académicos, el envío de información a servicios externos puede estar sujeto a restricciones de privacidad o políticas internas. La segunda es la reproducibilidad y continuidad: los modelos propietarios pueden cambiar por actualizaciones o interrupciones del servicio. La tercera es la evolución del ecosistema: una arquitectura con modelo parametrizado reduce el esfuerzo de adopción de nuevas variantes y mantiene estable el resto del pipeline.

En la práctica, la implementación actual soporta dos proveedores: OpenAI y Ollama. Esta decisión permite cubrir tanto el uso de servicios externos como la ejecución local de modelos de código abierto en contextos donde se prioriza la soberanía de datos. La incorporación de un proveedor adicional requiere implementar su adaptador correspondiente.

3.2. Selección de la Herramienta de Análisis Sintáctico

La validación sintáctica de las traducciones IBDD generadas por el LLM requiere una herramienta de análisis sintáctico (*parsing*) capaz de verificar

conformidad con la gramática formal IBDD. El analizador debe resolver ambigüedades sintácticas, proveer reportes de errores precisos, integrarse con Python (lenguaje de implementación del sistema), ofrecer confiabilidad para uso en investigación, y permitir la especificación declarativa de la gramática en un formato legible.

Lark [16] es una biblioteca de análisis sintáctico para Python que ha alcanzado amplia adopción tanto en la industria como en la academia. Implementa tres algoritmos principales de análisis. El algoritmo Earley logra rendimiento lineal en casos favorables y puede procesar gramáticas ambiguas, lo que lo hace adecuado para prototipado y lenguajes de estructura compleja; su principal limitación es un rendimiento aproximadamente diez veces inferior al de LALR en gramáticas no ambiguas [25]. El algoritmo LALR ofrece el mejor rendimiento para gramáticas no ambiguas bien definidas, aunque su *lexer* contextual puede presentar fallos en ciertos casos de ambigüedad [17]. El algoritmo CYK presenta limitaciones que incluyen un defecto reportado que causa bucles infinitos con gramáticas recursivas [17], por lo que no resulta apropiado para uso fuera de contextos teóricos.

Se consideraron otras bibliotecas de análisis sintáctico para Python antes de seleccionar Lark. PLY (*Python Lex-Yacc*) implementa exclusivamente LALR(1) y no soporta gramáticas ambiguas, lo que lo descarta para la gramática IBDD que combina estructuras EBNF con expresiones de lógica de primer orden. ANTLR4, si bien es una herramienta robusta con soporte para múltiples lenguajes, genera analizadores LL(*) que tampoco manejan ambigüedades de forma nativa, y su arquitectura basada en Java introduce una dependencia externa significativa. Las bibliotecas basadas en *Parsing Expression Grammars* (PEG), como pyparsing y TatSu, resuelven las ambigüedades mediante prioridad ordenada en lugar de explorar todas las derivaciones posibles, lo cual puede descartar silenciosamente interpretaciones válidas de la gramática IBDD. Lark, en cambio, ofrece el algoritmo Earley con soporte nativo para gramáticas ambiguas, especificación declarativa en EBNF y mensajes de error con posición exacta del fallo. Según análisis independientes [26], Lark se posiciona favorablemente frente a estas alternativas en el balance entre flexibilidad, rendimiento y facilidad de uso.

Lark está clasificado como *Production/Stable* en PyPI, cuenta con más de 5 500 estrellas en GitHub y mantiene desarrollo activo [23]. Además, presenta adopción en sistemas de escala empresarial: Dailymotion procesa más de 100 millones de llamadas API GraphQL diarias utilizando Lark para el análisis de *GraphQL Schema Definition Language* [5]; Vyper, el lenguaje de contratos inteligentes para Ethereum, emplea Lark para el análisis de su sintaxis [34]; y Poetry, el gestor de dependencias de Python, depende de Lark para procesar archivos de configuración [24].

En el ámbito académico, Hundvin [13] desarrolló en la Universidad de Bergen una tesis de maestría sobre generación de fragmentos de código a partir de especificaciones de gramática Lark, contribuyendo algoritmos para producir código sintácticamente correcto desde gramáticas formales. Este antecedente aporta evidencia de la pertinencia de Lark para investigación en análisis sintáctico y validación de lenguajes.

3.2.1. Decisión: Lark con Earley

La gramática IBDD presenta zonas potenciales de ambigüedad derivadas de la flexibilidad introducida en la implementación. En particular, la regla del *switch* acepta una secuencia flexible de guardas, expresiones y asignaciones tras la interacción, lo que obliga al analizador a decidir entre distintas derivaciones posibles para secuencias que mezclan condiciones, expresiones sueltas y actualizaciones de estado. Un analizador LALR requeriría desambiguar estas construcciones mediante transformaciones de la gramática que sacrificarían su legibilidad; el algoritmo Earley, en cambio, permite manejar esta flexibilidad de manera más directa.

El analizador Earley de Lark puede almacenar eficientemente todas las derivaciones posibles ante una ambigüedad [16]. La biblioteca ofrece tres estrategias de resolución: el modo `resolve` (predeterminado) elige automáticamente la derivación más simple; el modo `explicit` retorna todas las derivaciones para selección manual; y el modo `forest` retorna el bosque de análisis completo. En el presente trabajo se utiliza el modo `resolve`, que resulta adecuado para las ambigüedades de la gramática IBDD ya que la derivación más simple coincide con la interpretación semánticamente correcta en todos los casos observados.

Sobre esta base, se seleccionó Lark con el algoritmo Earley como herramienta de análisis sintáctico. Esta combinación ofrece la flexibilidad necesaria para manejar la estructura anidada y las potenciales ambigüedades de la gramática IBDD, facilita el ajuste y la extensión de la gramática durante el desarrollo, y cuenta con respaldo tanto industrial como académico. La gramática se especifica en formato EBNF, lo que favorece su lectura y mantenimiento, y el analizador provee información detallada sobre violaciones sintácticas que resulta útil para el ciclo de corrección iterativa.

El compromiso de rendimiento respecto a LALR (aproximadamente un orden de magnitud más lento) se considera aceptable dado que el volumen de escenarios a procesar es moderado y la corrección es prioritaria sobre la velocidad en esta fase de investigación. En un sistema de producción futuro, si el rendimiento pasara a ser un requisito dominante, sería razonable migrar a un analizador LALR.

3.3. Diseño del Prompt de Traducción

El diseño del prompt constituye un componente central del sistema, ya que determina la capacidad del modelo de lenguaje para producir traducciones IBDD sintáctica y semánticamente correctas. Se adoptó una estrategia de *few-shot prompting* con instrucciones estructuradas, organizada en cinco componentes que proveen al modelo con el contexto completo necesario para la tarea de traducción.

3.3.1. Estructura del Prompt

El prompt se organiza en cinco secciones. La primera establece el contexto general, indicando que se traducirán escenarios Gherkin a IBDD y que este lenguaje intermedio agrega precisión formal mediante sistemas de transición simbólicos. La segunda sección incluye la gramática IBDD completa en notación EBNF, junto con la descripción de los elementos clave del lenguaje: variables locales y globales, variables de interacción, gates de entrada, denotados con el prefijo `?`, y gates de salida, denotados con el prefijo `!`, guardas booleanas y asignaciones, así como el repertorio de operadores aceptados en notación Unicode y ASCII. La tercera sección provee ejemplos de traducción resueltos con explicaciones del razonamiento aplicado. La cuarta sección especifica reglas de traducción para cada cláusula (Given, When, Then) y convenciones de nomenclatura. La quinta sección define el formato de entrada y salida esperado y las restricciones de formato del resultado.

La inclusión de la gramática formal directamente en el prompt responde a un requisito fundamental: el modelo debe conocer la sintaxis exacta del lenguaje objetivo para generar traducciones conformes. Sin esta especificación, las traducciones presentarían las desviaciones sintácticas observadas durante la evaluación comparativa de modelos, donde incluso GPT-4o introducía variaciones cuando el prompt no era suficientemente explícito respecto a las reglas de formato.

El prompt completo utilizado en los experimentos puede encontrarse en el archivo `docs/PROMPT_ES.md` del repositorio y se reproduce en el Apéndice A.

3.3.2. Aprendizaje por Ejemplos

Se adoptó la técnica de *few-shot prompting* con cuatro ejemplos canónicos que cubren los patrones más frecuentes de la traducción BDD→IBDD. Estos cuatro ejemplos forman parte de la sección de ejemplos del prompt de traducción, que puede encontrarse en `docs/PROMPT_ES.md`. Los primeros tres provienen del dominio de impresora industrial, extraído de los trabajos

de Zameni et al. [36], y el cuarto del dominio de comercio electrónico. Esta selección de dos dominios distintos busca que el modelo generalice los patrones de traducción en lugar de memorizar un formato específico de dominio.

El primer ejemplo ilustra un escenario con una única variable local declarada en la cláusula Given, precondición trivial (`[true]`), un gate de entrada en When y un gate de salida en Then con postcondición de verificación. El segundo ejemplo presenta variables locales con guardas no triviales y dos switches secuenciales en When que modelan un proceso de dos etapas. El tercer ejemplo muestra un escenario sin variables locales donde la cláusula Given contiene únicamente una guarda sobre el estado global del sistema. El cuarto ejemplo introduce múltiples variables de interacción y asignaciones compuestas que actualizan más de un atributo del estado.

Cada ejemplo incluye el escenario Gherkin original en formato JSON, la traducción IBDD correspondiente y una explicación del razonamiento que justifica las decisiones de traducción. Esta estructura permite que el modelo observe no solo el resultado esperado sino también el proceso inferencial que lo produce. La elección de *few-shot* frente a *zero-shot* es consistente con los hallazgos reportados por Karpurapu et al. [15] en la literatura sobre automatización de BDD.

3.3.3. Reglas de Traducción

El prompt especifica reglas explícitas para la traducción de cada cláusula Gherkin. Para la cláusula Given, el modelo debe identificar si el texto menciona entidades que constituyen variables locales (por ejemplo, "a job file" se traduce a la variable local `JF`) o si describe condiciones sobre el estado global del sistema, que se modelan como guardas sin declaración de variables. Múltiples condiciones se combinan mediante el operador de conjunción lógica y la guarda resultante se encierra entre corchetes.

Para la cláusula When, las acciones del usuario se modelan como gates de entrada (`?`) y las acciones del sistema como gates de salida (`!`). Cuando el texto contiene múltiples acciones conectadas por "AND", cada acción se traduce como un switch independiente. En la forma canónica del lenguaje, cada switch incluye una condición entre corchetes y, opcionalmente, asignaciones que reflejan cambios de estado; en la implementación, la gramática admite además variantes más flexibles para tolerar salidas imperfectas del modelo durante la validación.

Un caso representativo es el segundo ejemplo canónico del prompt, donde la cláusula When contiene dos acciones consecutivas y se traduce como dos switches secuenciales:

```
WHEN !printstart.cj
  cj.id = CJ.id ∧ cj.state = Printing
  CJ.state := cj.state
!printcomplete.cj
  cj.id = CJ.id ∧ cj.state = Completed
  CJ.state := cj.state, PJL := add(CJ, PJL)
```

Para la cláusula Then, las verificaciones del estado final se expresan como postcondiciones en la guarda final, mientras que las acciones explícitas del sistema se modelan como switches con gates de salida. Las convenciones de nomenclatura establecen que las variables locales se representan como abreviaturas en mayúscula (`CJ`, `JF`, `P`), las variables globales como nombres descriptivos en mayúscula (`SJL`, `PJL`), las variables de interacción en minúscula (`cj`, `jf`, `p`) y los gates como verbos descriptivos en minúscula (`submit`, `printstart`, `add_to_cart`).

3.3.4. Formato de Intercambio

Se seleccionó JSON como formato de intercambio entre el sistema y el modelo de lenguaje, tanto para la representación de los escenarios BDD de entrada como para las traducciones IBDD de salida. Esta decisión se apoya en dos consideraciones. La primera es de compatibilidad técnica: JSON es un formato ampliamente utilizado en software, APIs y datos estructurados, y su sintaxis es simple, estandarizada y directamente procesable por las bibliotecas del pipeline. La segunda es empírica: en la práctica, los LLMs contemporáneos muestran buen desempeño en tareas de generación estructurada cuando se solicita salida en JSON, especialmente cuando la generación se combina con restricciones de esquema en la interfaz del proveedor [22].

Adicionalmente, desde el punto de vista de la representación textual, JSON ofrece regularidades sintácticas (llaves, corchetes, comillas y separadores) que se alinean bien con los esquemas de tokenización por subpalabras utilizados por los LLMs, como *Byte Pair Encoding* (BPE) [28]. Si bien este trabajo no mide de manera directa el efecto de la tokenización sobre distintos formatos de serialización, se adopta JSON como criterio de diseño por su robustez práctica, su facilidad de validación automática y su integración natural con salidas estructuradas y procesamiento posterior en Python.

El formato de entrada representa cada escenario BDD como un objeto JSON con campos `id`, `domain`, `title`, `given`, `when` y `then`. El formato de salida replica esta estructura incorporando un campo adicional, ibdd_representation, que contiene la traducción IBDD como una cadena de texto con saltos de línea (`\n`) para separar las cláusulas del escenario. Esta correspondencia estructural

entre entrada y salida facilita la trazabilidad entre el escenario original y su traducción formal, y permite que el sistema preserve los metadatos del escenario (dominio, título) a lo largo del pipeline de procesamiento.

3.4. Construcción del Dataset

La evaluación del sistema de traducción requiere un conjunto de escenarios BDD que cubra diversidad de dominios, estructuras sintácticas y niveles de complejidad. Dado que no conseguimos un corpus público de escenarios BDD anotados con traducciones IBDD de referencia, fue necesario construir un dataset específico para este trabajo.

El dataset resultante contiene 120 escenarios BDD distribuidos en cinco dominios de aplicación y tres niveles de complejidad. La Tabla 3.2 resume su composición.

En términos de organización de archivos, esta colección consolidada se almacena como `data/Dataset.json`. Para la evaluación experimental (Capítulo 5) se adopta una partición fija derivada de este archivo: un subconjunto de desarrollo de 30 escenarios, disponible en `data/Dataset_dev_30.json`, y un subconjunto de prueba de 90 escenarios, disponible en `data/Dataset_test_90.json`. El repositorio incluye además las versiones en español `data/Dataset_ES.json`, `data/Dataset_ES_dev_30.json` y `data/Dataset_ES_test_90.json`.

Tabla 3.2: Composición del dataset de escenarios BDD

Dominio	**Simple**	**Medio**	**Complejo**	**Total**
Impresora	8	8	8	24
Calculadora	12	8	4	24
Comercio electrónico	7	8	9	24
Banco	7	8	9	24
Biblioteca	7	8	9	24
Total	41	40	39	120

La selección del tamaño y la distribución del dataset responde a criterios estadísticos. Con 120 escenarios, cada caso individual representa el 0,83 % de la tasa de validez sintáctica (SVR), lo que proporciona una granularidad suficiente para detectar diferencias entre configuraciones experimentales. El diseño de cinco dominios y tres niveles de complejidad genera 15 celdas en la matriz dominio×complejidad, con un mínimo de 7 y un promedio de 8 escenarios por celda. Este tamaño permite realizar análisis de estabilidad por subgrupo con intervalos de confianza razonables y aplicar pruebas estadísticas

no paramétricas para comparaciones entre configuraciones.

La etiqueta de complejidad de cada escenario se definió mediante criterios estructurales explícitos, para evitar una clasificación subjetiva. En particular, se consideró la cantidad de entidades del dominio involucradas (objetos/actores que luego se reflejan en variables), el número de pasos en Given/When/Then, la cantidad de interacciones y condiciones, y la presencia de composiciones lógicas o asignaciones múltiples. Bajo este criterio, los escenarios complejos concentran mayor densidad de entidades y relaciones entre ellas, además de estructuras de decisión más largas que las de los casos simples y medios.

3.4.1. Escenarios Canónicos de Referencia

El punto de partida para la construcción del dataset fueron los escenarios canónicos provenientes de dos dominios: el dominio de impresora industrial, extraído de los trabajos de Zameni et al. [36], y el dominio de calculadora. Estos escenarios cuentan con traducciones IBDD verificadas manualmente y cubren los patrones fundamentales del lenguaje: declaración de variables locales con y sin guardas, switches con gates de entrada y salida, asignaciones simples y compuestas, y postcondiciones con funciones de verificación. Su inclusión en el dataset cumple una doble función: sirven como ejemplos de referencia dentro del prompt de traducción y como casos de validación cuya traducción correcta es conocida de antemano.

3.4.2. Generación Sintética de Escenarios

Los escenarios canónicos de referencia, si bien cubren los patrones fundamentales de IBDD, resultan insuficientes para una evaluación robusta del sistema. Un dataset de pocos escenarios concentrados en dos dominios no permite evaluar la capacidad de generalización del traductor ni proporciona la potencia estadística necesaria para comparar configuraciones experimentales. Para abordar esta limitación, se aplicó una estrategia de generación sintética de datos [7], utilizando un LLM para producir nuevos escenarios BDD a partir de los canónicos.

El proceso de ampliación del dataset se organizó en dos etapas. En la primera, se expandieron los dominios existentes para equilibrar la distribución de complejidad: el dominio de calculadora, que contenía casi exclusivamente escenarios simples, se complementó con escenarios de complejidad media y alta; el dominio de impresora se completó con escenarios simples adicionales. En la segunda etapa, se incorporaron tres nuevos dominios de aplicación: comercio electrónico, banco y biblioteca. Estos dominios fueron seleccionados por representar sistemas de software con patrones de interacción diferenciados

entre sí: gestión de inventario y pagos, operaciones financieras con autorizaciones y cálculos numéricos, y administración de recursos compartidos con reservas y políticas temporales.

La generación se realizó mediante prompting al LLM, proveyendo los escenarios canónicos como ejemplos de referencia junto con instrucciones para producir nuevos escenarios en formato Gherkin. Para esta etapa se utilizó *Claude Sonnet 4* de Anthropic, uno de los modelos analizados en la comparación inicial de la Sección 3.1. Los escenarios generados se representaron como objetos JSON con la estructura estándar del sistema (campos `id`, `domain`, `title`, `given`, `when`, `then`, `complexity`) y pueden encontrarse en `data/Dataset.json`, junto con sus particiones `data/Dataset_dev_30.json` y `data/Dataset_test_90.json`. Se solicitó que los escenarios variaran en la cantidad de pasos, el tipo de interacciones y la presencia de precondiciones y postcondiciones compuestas, manteniendo la plausibilidad funcional de las situaciones descritas. Para cada dominio se generaron 24 escenarios con distribución aproximadamente uniforme entre los tres niveles de complejidad.

Se decidió utilizar *Claude Sonnet 4*, un modelo distinto del empleado en la etapa principal de traducción y corrección (GPT-4o), para reducir el acoplamiento entre generación de datos y evaluación del sistema. Esta separación metodológica ayuda a mitigar sesgos por dependencia de estilo de un mismo modelo y disminuye el riesgo de sobreestimar el desempeño del traductor sobre casos sintéticos producidos con ese mismo estilo.

La utilización de un LLM como herramienta de generación sintética permite obtener escenarios con variaciones naturales en la redacción de las cláusulas Given, When y Then. Esta diversidad lingüística resulta relevante porque los escenarios BDD en entornos reales exhiben una variabilidad considerable en estilo y nivel de detalle según el autor, y el sistema de traducción debe ser capaz de manejar formulaciones heterogéneas de un mismo tipo de interacción. Como contrapartida, la generación sintética introduce el riesgo de producir escenarios que, si bien son gramaticalmente correctos en Gherkin, describen situaciones funcionalmente implausibles o ambiguas. Para mitigar este riesgo, cada escenario generado fue revisado manualmente, verificando su plausibilidad funcional, la coherencia entre las cláusulas Given, When y Then, y la ausencia de ambigüedades que pudieran afectar la traducción. Los escenarios que no cumplieron estos criterios fueron descartados o reformulados.

3.4.3. Dataset en Español

La literatura sobre BDD y los repositorios públicos de escenarios Gherkin están dominados por el idioma inglés. Los trabajos de Zameni et al. [36, 37, 38] emplean exclusivamente escenarios en inglés, al igual que los estudios

de Karpurapu et al. [15] sobre generación de tests BDD con LLMs. Esta predominancia del inglés se extiende a las herramientas del ecosistema BDD: si bien Cucumber soporta palabras clave en múltiples idiomas, la gran mayoría de los ejemplos, tutoriales y documentación se encuentran en inglés.

Sin embargo, una porción significativa de la comunidad de desarrollo de software opera en español, y las especificaciones de comportamiento en entornos hispanohablantes se redactan frecuentemente en este idioma. Un sistema de traducción BDD→IBDD que solo funcione con escenarios en inglés tendría una aplicabilidad limitada en estos contextos. Por este motivo, se construyó una versión en español del dataset BDD, traduciendo los escenarios del dataset original y verificando que las traducciones preserven estructura e intención sin introducir ambigüedades.

La inclusión de un dataset bilingüe permite evaluar el comportamiento del sistema ante escenarios en español. En la literatura relevada para este trabajo, no se encontraron reportes de evaluación BDD→IBDD en idioma español. Asimismo, es razonable esperar mejor desempeño con escenarios en inglés, dado que los LLMs suelen entrenarse con mayor proporción de texto en ese idioma y que la gramática IBDD emplea palabras clave en inglés (`GIVEN`, `WHEN`, `THEN`). En este marco, la evaluación bilingüe permite cuantificar esa diferencia y estimar el nivel de desempeño del sistema sobre el dataset en español.

La versión en español del dataset contiene los mismos 120 escenarios traducidos, preservando la estructura, la intención y el nivel de complejidad de cada caso. La traducción fue realizada de forma asistida y verificada manualmente para evitar ambigüedades introducidas por el cambio de idioma.

Capítulo 4

Implementación

4.1. Arquitectura del Sistema

El sistema implementa el pipeline de cuatro etapas orquestado por la clase `BDDToIBDDPipeline`, definida en el módulo principal. Cada etapa consume la salida de la anterior y guarda sus resultados en archivos JSON intermedios, lo que permite ejecutar las etapas de manera independiente durante el desarrollo y la depuración.

1. **Traducción**. Recibe el archivo `Dataset.json` con los escenarios BDD en formato Gherkin y produce traducciones IBDD mediante un LLM, utilizando salidas estructuradas para garantizar conformidad con el esquema JSON esperado. Las traducciones se guardan incrementalmente en `output.json`. En la etapa experimental, este mismo mecanismo se aplica también a subconjuntos derivados del dataset consolidado (por ejemplo, particiones de desarrollo y prueba), manteniendo el mismo formato de entrada. Esta etapa se describe en la Sección 4.3.

2. **Validación sintáctica**. Cada traducción se analiza contra la gramática formal IBDD utilizando el analizador Earley de Lark. Los escenarios que no superan la validación se identifican con información detallada del error, incluyendo la posición y el tipo de violación sintáctica. Los resultados se almacenan en `parsed_ibdd_results.json`. Esta etapa se describe en la Sección 4.4.

3. **Análisis de errores**. Los escenarios que fallaron en la validación sintáctica son procesados por un agente especializado, el *explainer*, que diagnostica la causa del error y genera una explicación estructurada con sugerencias de corrección. Esta etapa se describe en la Sección 4.5.

4. **Reintento con correcciones**. Las explicaciones de error se inyectan en un prompt de reintento que solicita al LLM una nueva traducción informada por el diagnóstico previo. Las traducciones corregidas reemplazan únicamente los casos fallidos mediante una estrategia de fusión no destructiva, y el ciclo de validación se repite sobre los casos retraducidos. Esta etapa se describe en la Sección 4.5.

Como se indicó al comienzo de esta sección, los datos fluyen secuencialmente desde `Dataset.json` (o desde particiones derivadas con la misma estructura) hasta los archivos de resultados finales, y todas las etapas operan sobre archivos JSON como formato de intercambio.

4.2. Definición de la Gramática IBDD

La gramática IBDD se especifica en formato EBNF compatible con Lark y constituye el componente central del módulo de validación. Esta es la gramática operacional adoptada por el sistema: toma como base la estructura general del lenguaje IBDD presentada en la Sección 2.4 y la adapta para manejar las variaciones de formato que genera el LLM durante la traducción.

4.2.1. Estructura de la Gramática

La gramática define un escenario IBDD como la secuencia de tres cláusulas obligatorias: Given, When y Then. El Listado 4.1 presenta las reglas estructurales principales.

Listing 4.1 Reglas estructurales de la gramática IBDD en Lark

```
scenario: given when then

given: "GIVEN" [vars] guard
vars: var (COMMA var)*

when: "WHEN" switch+
then: "THEN" switch* guard

switch: interaction (guard | expr | assignment)*
interaction: gate [DOT var_list]
gate: /[!?][a-zA-Z][a-zA-Z0-9_]*/
var_list: var (COMMA var)*

guard: LBRACKET expr RBRACKET
```

La cláusula `given` acepta opcionalmente una lista de variables locales seguida de una guarda entre corchetes. La cláusula `when` requiere al menos un switch, mientras que `then` acepta cero o más switches seguidos de una guarda final que representa la postcondición. Cada switch se compone de una interacción (un gate con dirección `?` para entrada o `!` para salida, y opcionalmente una lista de variables de interacción) seguida de una secuencia flexible de guardas, expresiones y asignaciones.

La regla `switch` merece atención particular. Su definición emplea una alternancia con repetición:

```
switch: interaction (guard | expr | assignment)*
```

Esta formulación permite acomodar las diversas formas en que el LLM estructura los switches. En la gramática teórica de IBDD, un switch sigue estrictamente el patrón *interacción – condición – asignación*; sin embargo, las traducciones generadas por el modelo pueden omitir la condición, presentar la asignación antes de la guarda, o incluir expresiones sin envolver en corchetes. Esta flexibilidad sintáctica es una decisión de diseño deliberada que prioriza la capacidad de analizar traducciones imperfectas sobre el rechazo estricto de variaciones menores.

4.2.2. Jerarquía de Expresiones

Las expresiones de la gramática se organizan en una jerarquía de precedencia que refleja las convenciones matemáticas estándar. El Listado 4.2 muestra esta jerarquía.

Listing 4.2 Jerarquía de expresiones en la gramática IBDD

```
expr: or_expr
or_expr: and_expr ((OR | OR_SYMBOL) and_expr)*
and_expr: not_expr ((AND | AND_SYMBOL) not_expr)*
not_expr: (NOT | NOT_SYMBOL) not_expr | comparison
comparison: sum [op sum]
sum: product ((PLUS|MINUS) product)*
product: power ((STAR|SLASH|PERCENT) power)*
power: atom [CIRCUMFLEX atom]
atom: literal | var | func_call
     | prop_access | neg_number
     | LPAR expr RPAR
```

La disyunción tiene la menor precedencia, seguida por la conjunción, la negación, las comparaciones, las operaciones aritméticas de suma y resta, las de multiplicación, división y módulo, y finalmente la exponenciación. Los átomos constituyen las unidades indivisibles de las expresiones e incluyen literales booleanos (`true`, `false`) y numéricos, variables, llamadas a funciones (como `is_in_list(CJ, SJL)`), accesos a propiedades (`CJ.type`) y expresiones parentizadas.

4.2.3. Soporte de Operadores

La gramática acepta tanto notación Unicode como ASCII para los operadores lógicos y de comparación. La conjunción puede expresarse como $\wedge$ o `&&`, la disyunción como $\vee$ o `||`, y la negación como $\neg$ o `!`. De manera similar, la desigualdad admite $\neq$ o `!=`, y los operadores de orden $\leq$/`<=` y $\geq$/`>=`. Este soporte dual responde a la variabilidad observada en las salidas del LLM, que puede utilizar caracteres Unicode o sus equivalentes ASCII según el contexto de generación. Sin esta flexibilidad, traducciones semánticamente correctas serían rechazadas por el analizador debido a diferencias puramente notacionales.

Durante las iteraciones de desarrollo y las pruebas preliminares se observó, además, que una parte relevante de los fallos de parsing no provenía de una mala correspondencia semántica con el escenario BDD, sino del uso de

variantes de notación no contempladas inicialmente por la gramática. Este patrón motivó la incorporación de equivalencias sintácticas seguras (por ejemplo, operadores lógicos y de comparación en notación ASCII y Unicode) cuando la variación no altera la interpretación del término IBDD.

4.2.4. Transformación del Árbol Sintáctico

El análisis sintáctico produce un árbol de derivación que se transforma en objetos del dominio mediante la clase `IBDDTransformer`, implementada como un transformador de Lark en el archivo `src/parser.py`, donde también se definen las clases de datos `IBDDScenario`, `IBDDSwitch`, `IBDDInteraction`, `IBDDAssignment` e `IBDDExpression`. La transformación mapea cada nodo del árbol a una de estas cinco clases: `IBDDScenario` representa el escenario completo con variables locales, precondición, switches de When y Then, y postcondición; `IBDDSwitch` encapsula una interacción con su condición y asignaciones; `IBDDInteraction` modela un gate con sus variables; `IBDDAssignment` representa una asignación de valor a variable o propiedad; e `IBDDExpression` captura expresiones tipadas con sus subexpresiones en una estructura recursiva. Esta representación intermedia facilita la inspección de las traducciones y la generación de reportes de validación estructurados.

4.3. Módulo de Traducción

El módulo de traducción se implementa en la clase `TranslationService` y gestiona la comunicación con el LLM para producir traducciones IBDD a partir de escenarios BDD. El diseño del módulo se organiza en torno a cuatro aspectos: la abstracción del proveedor de LLM, la garantía estructural de las respuestas mediante salidas estructuradas, la inferencia dinámica del esquema JSON, y la estrategia de procesamiento de escenarios.

4.3.1. Abstracción del Proveedor de LLM

El sistema se diseñó para evitar el acoplamiento a un modelo puntual y para operar con proveedor y modelo parametrizados desde configuración. Esta separación se implementa mediante la clase `LLMClient`, que encapsula las diferencias entre proveedores detrás de una interfaz uniforme. El cliente recibe como parámetros el identificador del proveedor (`openai`, `ollama`), el nombre del modelo, la clave de API (cuando corresponde) y la temperatura de generación, y expone un único método `generate_json` que acepta un prompt de sistema, un prompt de usuario y un esquema JSON de respuesta.

Para el proveedor OpenAI, el cliente utiliza la biblioteca LangChain con el conector `ChatOpenAI` e invoca el mecanismo de *Structured Outputs* con esquema estricto (`strict: true`), que garantiza conformidad estructural de la respuesta. Para proveedores locales como Ollama, el cliente utiliza el conector `ChatOllama` con formato JSON y aplica validación de esquema *a posteriori* mediante la biblioteca `jsonschema`; en caso de que la respuesta no cumpla con el esquema, el cliente reintenta la solicitud hasta un máximo configurable de intentos con *exponential backoff*[1].

Esta arquitectura permite que el resto del pipeline (el servicio de traducción, el analizador de errores y el orquestador principal) opere sin conocimiento del proveedor subyacente dentro del conjunto de proveedores actualmente implementados. La selección del modelo y del proveedor se realiza mediante parámetros de línea de comandos (`--provider`, `--model`) o variables de entorno (`LLM_PROVIDER`, `LLM_MODEL`), lo que facilita la evaluación comparativa entre modelos y la adaptación a diferentes contextos de despliegue sin modificar el código fuente. La incorporación de nuevos proveedores requiere extender la clase `LLMClient` con un adaptador adicional, debido a diferencias entre APIs (por ejemplo, soporte nativo de salidas estructuradas estrictas).

4.3.2. Structured Outputs de OpenAI

El sistema utiliza la funcionalidad de *Structured Outputs* de OpenAI [22], que permite especificar un esquema JSON (*JSON Schema*) como parte de la solicitud a la API. Cuando se activa el modo `json_schema` con la opción `strict: true`, la API garantiza que la respuesta del modelo se ajustará exactamente al esquema proporcionado. Esta garantía se implementa mediante *constrained decoding* [22]: durante la generación, el modelo descarta las secuencias de tokens que violarían el esquema antes de seleccionar el siguiente token, asegurando que el resultado contenga exactamente los campos especificados con los tipos correctos.

El uso de Structured Outputs en este sistema presenta dos ventajas concretas. La primera es la eliminación de errores de formato: sin esta restricción, el modelo podría devolver JSON malformado, incluir campos adicionales no solicitados, o envolver la respuesta en bloques de código Markdown, lo cual requeriría lógica de post-procesamiento frágil y propensa a errores. La segunda ventaja es la garantía de presencia del campo `ibdd_representation`, que es donde el modelo deposita la traducción IBDD para cada escenario. Al declarar este campo como obligatorio en el esquema, se asegura que toda

[1] El *exponential backoff* incrementa el tiempo de espera entre reintentos (típicamente de forma exponencial), reduciendo colisiones y sobrecarga ante fallos transitorios [19].

respuesta válida contenga la traducción solicitada.

Structured Outputs garantiza la corrección estructural del JSON pero no la corrección sintáctica del contenido IBDD. El campo `ibdd_representation` contiene una cadena de texto cuyo contenido, la traducción IBDD, requiere validación adicional mediante el analizador sintáctico descrito en la Sección 4.2. Esta distinción entre validación de formato (a cargo de Structured Outputs) y validación de contenido (a cargo del parser Earley) es fundamental en la arquitectura del sistema.

4.3.3. Inferencia Dinámica del Esquema

El esquema JSON que se envía a la API no se define estáticamente, sino que se infiere dinámicamente a partir de la estructura de los datos de entrada mediante el método `create_response_schema`. Este método recorre recursivamente el JSON de entrada, determina el tipo de cada campo (`string`, `integer`, `boolean`, `array`, `object`) y construye un esquema compatible con JSON Schema que replica la estructura observada. Adicionalmente, el método inyecta automáticamente el campo `ibdd_representation` de tipo `string` si este no está presente en los datos de entrada, asegurando que el esquema resultante siempre incluya el campo de salida requerido para la traducción.

Para entradas de tipo arreglo (el caso habitual, donde se procesa un conjunto de escenarios), el esquema resultante envuelve los elementos en un objeto con un campo `items` de tipo arreglo. Esta envoltura responde a un requisito técnico de la API de OpenAI, que exige que el esquema raíz sea de tipo objeto. El servicio de traducción deshace esta envoltura al recibir la respuesta, extrayendo de manera transparente el arreglo de escenarios traducidos.

La inferencia dinámica permite que el sistema se adapte a variaciones en la estructura del *dataset* de entrada sin requerir modificaciones en el código. Si se agregan campos al JSON de entrada (por ejemplo, notas o metadatos adicionales), el esquema inferido los incorporará automáticamente y el modelo los preservará en la salida.

4.3.4. Procesamiento de Escenarios

El servicio procesa los escenarios de manera individual: cada escenario BDD se envía como una solicitud independiente a la API y su traducción se guarda de manera incremental en el archivo de salida. Esta estrategia de procesamiento individual, en contraposición al envío de todos los escenarios en una única solicitud, responde a dos consideraciones.

La primera es el control del tamaño del contexto por solicitud. El envío del dataset completo en una única solicitud implicaría un prompt de entrada de gran tamaño, ya que cada escenario adicional incrementa el número de tokens de contexto. El procesamiento individual minimiza el contexto por solicitud y mantiene acotada la longitud de la respuesta en cada interacción. Además, las solicitudes con contextos extensos presentan mayor probabilidad de respuestas truncadas o degradadas en calidad cuando el modelo debe mantener coherencia a lo largo de una salida muy extensa.

La segunda consideración es la resiliencia ante interrupciones: dado que cada traducción se guarda de manera incremental en el archivo de salida, si el proceso se detiene por un error de red, una limitación temporal del servicio u otra causa, las traducciones completadas se conservan y no es necesario reprocesar todo el conjunto de datos.

El módulo implementa un mecanismo de reintentos con *exponential backoff* para manejar errores transitorios de la API, particularmente los errores de límite de tasa (código HTTP 429). Ante un error, el sistema espera un intervalo que se duplica en cada reintento ($t_i = t_{\text{base}} \cdot 2^{i-1} + \epsilon$, donde ϵ es una perturbación aleatoria uniforme), hasta un máximo de cinco intentos. Este mecanismo permite procesar conjuntos de datos extensos sin interrupciones por limitaciones temporales de la API.

La temperatura del modelo se configura en 0,7, seleccionada como valor intermedio tras pruebas preliminares en desarrollo y en línea con la literatura que reporta que la temperatura afecta el compromiso entre variabilidad y consistencia de salida [12, 27]. En este trabajo, este valor se adopta para mantener diversidad suficiente entre corridas sin perder estabilidad sintáctica. El modelo utilizado es configurable mediante parámetros de línea de comandos, lo que permite evaluar diferentes versiones del modelo sin modificar el código fuente.

4.4. Módulo de Validación Sintáctica

El módulo de validación instancia la clase `IBDDParser`, que encapsula el analizador Earley de Lark. El parser se configura con dos opciones relevantes: resolución automática de ambigüedades (`ambiguity='resolve'`) y propagación de posiciones (`propagate_positions=True`). La primera opción indica al analizador que seleccione automáticamente la derivación más simple ante una ambigüedad, evitando la necesidad de intervención manual. La segunda habilita la generación de mensajes de error que incluyen la línea y columna exactas donde se detectó la violación sintáctica, información útil para diagnosticar problemas en las traducciones.

Antes del análisis, cada texto IBDD se somete a un preprocesamiento que normaliza los caracteres de escape (`\n` a saltos de línea reales), elimina líneas vacías y colapsa espacios múltiples dentro de cada línea. Este preprocesamiento es necesario porque las traducciones generadas por el LLM representan los saltos de línea como secuencias de escape dentro de cadenas JSON, y el analizador Earley requiere el texto con saltos de línea reales para aplicar las reglas de la gramática que dependen de la separación de cláusulas.

La validación opera en modo por lotes: el sistema procesa secuencialmente todos los escenarios de un archivo JSON, registrando para cada uno si el análisis fue exitoso o, en caso de fallo, el mensaje de error del analizador. Los resultados se registran en un archivo JSON que asocia cada escenario con su estado de validación (`valid: true/false`), el dominio, el título y, cuando corresponde, el mensaje de error y la representación parseada del escenario. Este informe estructurado sirve como insumo para las etapas posteriores.

El módulo incluye además un analizador de respaldo (*fallback parser*) basado en expresiones regulares que se activa cuando el analizador Earley no logra procesar un texto IBDD. Este analizador alternativo extrae los componentes principales del escenario (variables de la cláusula Given, gates de la cláusula When y postcondición de la cláusula Then) mediante patrones de coincidencia, produciendo una representación parcial pero útil para el diagnóstico de errores.

4.5. Módulo de Corrección Iterativa

Cuando el analizador sintáctico detecta que una traducción IBDD no es conforme a la gramática, el pipeline activa un ciclo de corrección que involucra tres etapas: identificación de los casos fallidos, análisis automatizado de los errores mediante un agente explicador, y reintento de la traducción con contexto del error. Este ciclo constituye el mecanismo central de autocorrección del sistema y opera de manera no destructiva sobre las traducciones existentes.

4.5.1. Identificación de Casos Fallidos

Al finalizar la validación sintáctica, el orquestador del pipeline cruza los resultados de validación con las traducciones originales para construir un expediente completo de cada caso fallido. Para cada escenario cuyo campo `valid` es `false`, el sistema recopila cuatro elementos: el identificador del caso, el escenario BDD original (cláusulas Given, When y Then en lenguaje natural), la traducción IBDD que generó el error, y el mensaje de error producido por

el analizador Earley, que incluye la posición exacta (línea y columna) y el tipo de token inesperado. Esta información constituye el insumo que alimenta al agente explicador de errores.

4.5.2. Análisis de Errores

El análisis de errores se implementa en la clase `IBDDErrorExplainer`, definida en `src/explainer.py`, un agente especializado que utiliza el LLM para interpretar los mensajes de error del parser y producir diagnósticos estructurados. A diferencia del módulo de traducción, que opera con una temperatura de 0,7, el explicador se configura con una temperatura de 0,3. Este valor más bajo prioriza respuestas más estables y con mayor *grounding*[2], lo cual es apropiado para una tarea de diagnóstico donde la creatividad no es deseable [27].

El agente explicador opera con un prompt de sistema que incluye la gramática IBDD de referencia y un catálogo de patrones de error frecuentes: corchetes faltantes en guardas o condiciones, sintaxis de gate incorrecta (confusión entre `?` y `!`), asignaciones ausentes, expresiones mal formadas, saltos de línea incorrectos y postcondiciones faltantes. Este catálogo permite que el modelo enfoque su análisis en las categorías de error más probables.

Para cada caso fallido, el explicador construye un prompt de análisis que presenta al modelo el escenario BDD original, la traducción IBDD que falló y el mensaje de error del parser, solicitando un diagnóstico estructurado. La respuesta se obtiene mediante Structured Outputs con un esquema que define cuatro campos obligatorios: `error_type` (clasificación del error, por ejemplo "corchetes faltantes" o "gate inválido"), `error_location` (posición exacta dentro del texto IBDD), `explanation` (descripción de la discrepancia entre lo esperado por el parser y lo encontrado) y `correction_suggestion` (fragmento IBDD corregido o sugerencia concreta), además de un campo opcional `hints` (lista de recomendaciones adicionales). El uso de Structured Outputs garantiza la conformidad estructural del diagnóstico con este esquema, facilitando su procesamiento automatizado en la etapa siguiente.

4.5.3. Reintento con Contexto de Error

El reintento de traducción utiliza un prompt especializado que extiende el prompt de traducción inicial con información del error. El prompt de reintento contiene las mismas cinco secciones del prompt original (contexto, gramática, ejemplos, reglas y formato), pero incorpora al inicio una sección adicional

[2]En este trabajo, *grounding* refiere al grado de anclaje de la salida en la información explícita del contexto de entrada (escenario BDD, traducción fallida y mensaje del parser), minimizando inferencias no sustentadas por esos datos.

de análisis de error, precedida por un encabezado que indica al modelo que se encuentra en modo de corrección. Esta sección se inyecta dinámicamente mediante un *placeholder* `{error_analysis}` que el sistema reemplaza con el diagnóstico formateado del caso específico.

El formato del diagnóstico inyectado incluye el escenario BDD original, la traducción previa que falló, el error del parser, el análisis completo del explicador (tipo, ubicación, explicación y sugerencia de corrección) y las recomendaciones adicionales. De este modo, el modelo de traducción recibe el contexto completo del fallo y cuenta con contexto suficiente para aplicar la corrección. El prompt de reintento añade además una lista de verificación final que instruye al modelo a confirmar, antes de emitir la respuesta, que la corrección aborda el error específico identificado, que todos los corchetes están correctamente ubicados, que la sintaxis de gates es válida y que la traducción mantiene la correspondencia semántica con el escenario BDD original.

El procesamiento de reintentos opera de manera individual: cada caso fallido se reenvía como una solicitud independiente a la API, utilizando el mismo mecanismo de Structured Outputs y reintentos con *exponential backoff* del módulo de traducción inicial. Esta independencia permite que el fallo de un reintento no afecte la corrección de los demás casos.

4.5.4. Integración de Correcciones

Las traducciones corregidas se integran al conjunto de resultados mediante un procedimiento de reemplazo selectivo. El sistema carga las traducciones originales, construye un mapa indexado por identificador de caso con las traducciones corregidas, y recorre secuencialmente las traducciones originales: para cada caso cuyo identificador coincide con una corrección, se reemplaza la traducción original por la corregida; los casos que no presentaron errores se preservan intactos. Este diseño evita modificar las traducciones ya válidas durante el proceso de corrección.

Tras la fusión, el pipeline ejecuta una re-validación sintáctica sobre el archivo de traducciones actualizado, aplicando el mismo analizador Earley utilizado en la validación inicial. Esta re-validación cumple dos funciones: confirmar que las correcciones resolvieron efectivamente los errores de parsing, e identificar casos donde el reintento no logró producir una traducción conforme. Los resultados de la re-validación se escriben en el mismo archivo de validación, actualizando el estado de cada caso.

Las etapas de análisis de errores y reintento se clasifican como no críticas dentro del pipeline: si alguna de ellas falla (por ejemplo, por un error de la API o un agotamiento de la cuota), el pipeline continúa con las traducciones originales y emite una advertencia. Esta decisión se fundamenta en que es

preferible contar con traducciones parcialmente correctas (las exitosas de la primera pasada) que interrumpir todo el proceso por un fallo en la corrección de los casos problemáticos.

Capítulo 5

Evaluación

5.1. Preguntas de Investigación

La evaluación se organiza en torno a tres preguntas de investigación:

RQ1: *¿En qué medida el sistema genera traducciones IBDD sintácticamente válidas a partir de escenarios BDD?*

Esta pregunta evalúa la capacidad del sistema para producir traducciones que se ajusten a la gramática formal IBDD. Dado que la validación sintáctica es determinística (realizada por el parser Earley), la variabilidad en los resultados proviene exclusivamente de la naturaleza estocástica del LLM. Por ello, la respuesta a esta pregunta requiere múltiples ejecuciones del pipeline sobre el mismo dataset para cuantificar tanto la tasa de éxito promedio como su dispersión.

RQ2: *¿Cómo incide el idioma del prompt y del dataset en la calidad de la traducción?*

El sistema soporta prompts en inglés y en español, y los escenarios BDD pueden estar redactados en cualquiera de los dos idiomas. Esta pregunta investiga si la combinación de idiomas entre el prompt y el dataset afecta la tasa de validez sintáctica de las traducciones. La respuesta permite determinar si el sistema es robusto ante variaciones lingüísticas o si existe una configuración de idioma preferible.

RQ3: *¿Cuál es la efectividad del ciclo de corrección iterativo?*

El pipeline incluye un mecanismo de corrección que analiza los errores sintácticos de las traducciones fallidas y reintenta la traducción con retroalimentación del diagnóstico. Esta pregunta evalúa cuántas de

las traducciones inicialmente fallidas se corrigen mediante este ciclo, cuántas iteraciones son necesarias para converger y si existen casos que resisten la corrección de forma persistente.

5.2. Experimentos

5.2.1. Configuraciones

Para responder a las tres preguntas de investigación, se define un conjunto de configuraciones experimentales que combinan dos factores: el idioma del prompt de traducción y el idioma del dataset de escenarios BDD. Esto da lugar a un diseño factorial 2×2:

Tabla 5.1: Configuraciones experimentales

Configuración	**Prompt**	**Dataset**
EN–EN	Inglés	Inglés
ES–EN	Español	Inglés
EN–ES	Inglés	Español
ES–ES	Español	Español

La configuración EN–EN constituye la línea base, ya que tanto el prompt como el dataset fueron diseñados originalmente en inglés. Las configuraciones ES–EN y EN–ES aíslan el efecto del idioma del prompt y del dataset respectivamente, mientras que ES–ES evalúa el sistema operando íntegramente en español. Este diseño permite separar el efecto del idioma del prompt del efecto del idioma del dataset, y detectar si la coincidencia entre ambos idiomas incide en los resultados.

5.2.2. Partición del dataset (desarrollo y prueba)

Para evitar sesgos por ajuste sobre el mismo conjunto utilizado en la evaluación final, se adopta una partición fija del dataset de 120 escenarios en dos subconjuntos:

- **Desarrollo (dev):** 30 escenarios.
- **Prueba (test):** 90 escenarios.

La partición se definió de forma consistente con el historial de construcción del dataset: el subconjunto *dev* coincide con la versión inicial de 30 escenarios que fue utilizada durante las etapas tempranas de ajuste y depuración,

mientras que los 90 escenarios restantes conforman el conjunto *test* reservado para la evaluación final. En términos de archivos, esto se materializa en `Dataset_dev_30.json` y `Dataset_test_90.json`, así como sus equivalentes en español.

Una vez fijada la configuración del sistema sobre el conjunto *dev*, las corridas reportadas en la Sección 5.5 se realizarán exclusivamente sobre el conjunto *test*.

5.2.3. Repeticiones y manejo de la estocasticidad

Los LLMs son modelos estocásticos: ante la misma entrada y con temperatura mayor a cero, pueden producir salidas distintas en cada ejecución. Una única corrida del pipeline no permite distinguir si un resultado refleja el comportamiento típico del sistema o una fluctuación aleatoria. Para obtener estimaciones confiables de las métricas, cada configuración se ejecuta N veces de forma independiente sobre el conjunto *test*.

En la implementación experimental de este trabajo, N se parametriza desde línea de comandos y se registra automáticamente junto con los resultados agregados. Para la evaluación final se adopta inicialmente $N = 5$ corridas por configuración. Esta cantidad se considera un compromiso razonable entre obtener una estimación con variabilidad observable (evitando un número demasiado bajo de corridas, estadísticamente poco representativo) y mantener la ejecución dentro de los límites de tiempo y del alcance de este trabajo.

A partir de las N ejecuciones de cada configuración se calcula la media y la desviación estándar de cada métrica, y se construyen intervalos de confianza al 95 % mediante la distribución t de Student.

5.2.4. Ciclo de corrección

En todas las configuraciones, el pipeline se ejecuta con un número máximo de rondas de corrección configurable mediante el parámetro `--max-rounds`. En cada ronda, los casos que fallaron la validación sintáctica son procesados por el agente de análisis de errores y retraducidos con retroalimentación del diagnóstico. El ciclo se detiene cuando todos los casos superan la validación o cuando se alcanza el límite de rondas.

El valor de `--max-rounds` también se parametriza y queda registrado en los metadatos de cada ejecución para garantizar trazabilidad y reproducibilidad.

Este diseño permite evaluar la convergencia del ciclo de corrección: para cada ejecución se registra cuántos casos se corrigen en cada ronda, lo que permite construir una curva de convergencia promediada sobre las N repeticiones.

5.2.5. Protocolo de ejecución

Cada ejecución del pipeline sigue el protocolo descrito a continuación:

1. Se invoca el pipeline con la configuración correspondiente (prompt, dataset, modelo, número máximo de rondas).
2. El sistema traduce cada escenario BDD de forma individual, registrando el tiempo de traducción por caso.
3. Se valida sintácticamente cada traducción mediante el parser Earley.
4. Se ejecuta el ciclo de corrección iterativo hasta convergencia o agotamiento del límite de rondas.
5. Se guardan las traducciones finales, los resultados de validación y las métricas del pipeline en archivos JSON.

Previo a estas corridas de evaluación final sobre *test*, se realizan corridas de desarrollo sobre *dev* con el único objetivo de depuración y ajuste del enfoque. Dichas corridas no se incluyen en el reporte cuantitativo final para evitar que decisiones de ajuste informadas por esos resultados contaminen la evaluación reportada sobre el conjunto de prueba.

El pipeline registra automáticamente las métricas de cada ejecución en un archivo `*_metrics.json` que incluye: el modelo y proveedor utilizados, los resultados de validación iniciales y finales, el detalle por ronda de corrección (casos fallidos, casos corregidos, tiempo) y el tiempo total del pipeline.

5.3. Dataset

Para la evaluación se emplean dos versiones del dataset construido en la Sección 3.4: la versión original en inglés y su traducción al español, lo que permite ejecutar las cuatro configuraciones del diseño factorial.

El archivo consolidado contiene 120 escenarios (`Dataset.json` y su versión en español). Sin embargo, para el reporte de evaluación final se utiliza exclusivamente el subconjunto *test* de 90 escenarios (y su contraparte en español), mientras que el subconjunto *dev* de 30 escenarios queda reservado para ajuste y depuración.

La composición detallada del dataset consolidado se presenta en la Tabla 3.2 (Sección 3.4). En síntesis, el conjunto incluye 120 escenarios distribuidos en cinco dominios y tres niveles de complejidad (41 simples, 40 medios y 39 complejos), base sobre la cual se define la partición fija *dev/test*.

Como se mencionó previamente en la Sección 3.4, cada escenario está clasificado por dominio de aplicación y por nivel de complejidad. La complejidad se determina en función de la cantidad de entidades del dominio involucradas, la cantidad de pasos en las cláusulas Given, When y Then, la presencia de conjunciones (And) y la cantidad de variables e interacciones que el escenario implica. Los escenarios simples involucran una o dos interacciones con pocas variables, los de complejidad media presentan múltiples pasos con variables locales y globales, y los complejos combinan guardas condicionales, múltiples gates y asignaciones compuestas.

En coherencia con el protocolo metodológico descrito en la Sección 3.4, los escenarios sintéticos del dataset fueron generados con *Claude Sonnet 4*, mientras que la traducción y corrección evaluadas en este capítulo se ejecutan con GPT-4o. Esta separación busca evitar que la evaluación favorezca artificialmente a un único estilo de generación.

5.4. Métricas

La evaluación utiliza métricas que capturan aspectos complementarios del desempeño del sistema. Se distinguen tres categorías: métricas de validez sintáctica, métricas del ciclo de corrección y métricas operativas. El criterio compartido entre estas categorías es que todas pueden calcularse a partir de la salida del parser y los metadatos registrados por el pipeline, sin requerir anotación humana ni traducciones de referencia. Esto garantiza objetividad y reproducibilidad de los resultados, a costa de no capturar la dimensión semántica de las traducciones.

5.4.1. Validez sintáctica

La métrica principal es la *tasa de validez sintáctica* (SVR, *Syntactic Validity Rate*), definida como la proporción de escenarios cuya traducción IBDD es aceptada por el parser Earley descrito en la Sección 4.4:

$$\mathrm{SVR} = \frac{\text{número de traducciones sintácticamente válidas}}{\text{número total de escenarios}} \tag{5.1}$$

Se distinguen dos variantes de esta métrica. $\mathrm{SVR}_{\text{inicial}}$ mide la tasa de validez tras la primera traducción, sin correcciones. $\mathrm{SVR}_{\text{final}}$ mide la tasa de validez tras completar el ciclo de corrección iterativo. La diferencia $\mathrm{SVR}_{\text{final}} - \mathrm{SVR}_{\text{inicial}}$ cuantifica la ganancia atribuible al mecanismo de corrección.

Para cada configuración, se reportan la media y la desviación estándar de ambas variantes sobre las N ejecuciones independientes.

La elección de SVR como métrica principal se apoya en tres razones. En primer lugar, el resultado del parser Earley es determinístico ante una traducción dada: no interviene ningún juicio subjetivo en la medición, lo que garantiza reproducibilidad independientemente del evaluador. En segundo lugar, es directamente relevante para el objetivo del sistema, cuya función es producir traducciones conformes a la gramática IBDD. En tercer lugar, no requiere un corpus de traducciones de referencia anotadas manualmente, lo que resulta relevante dado que no existe un repositorio público de escenarios BDD con traducciones IBDD validadas.

5.4.2. Ciclo de corrección

Para evaluar la efectividad del ciclo de corrección (RQ3) se definen las siguientes métricas:

Tasa de recuperación: proporción de traducciones inicialmente fallidas que se corrigen tras el ciclo completo. Formalmente:

$$\mathrm{RR} = \frac{\text{fallidas iniciales} - \text{fallidas finales}}{\text{fallidas iniciales}} \tag{5.2}$$

Rondas hasta convergencia: número de rondas de corrección necesarias hasta que no se producen correcciones adicionales o se alcanza el límite configurado. Esta métrica indica la velocidad de convergencia del ciclo.

Correcciones por ronda: cantidad de casos corregidos en cada ronda individual. Permite construir una curva de convergencia que muestra si la mayoría de las correcciones ocurren en las primeras rondas o si se distribuyen de forma uniforme.

La inclusión de estas tres métricas responde a la necesidad de evaluar el ciclo de corrección de manera cuantitativa y complementaria: RR sintetiza el beneficio neto del mecanismo, las rondas hasta convergencia caracterizan su velocidad, y la curva de correcciones por ronda revela si las mejoras se concentran en las primeras iteraciones o requieren múltiples rondas. Sin este conjunto de métricas no sería posible determinar si el ciclo aporta un beneficio real o si los escenarios que fallan en la primera traducción continúan fallando independientemente del número de intentos.

5.4.3. Estabilidad por caso

Para identificar escenarios que presentan dificultad persistente para el sistema, se analiza la estabilidad de los resultados a nivel de caso individual. Para cada escenario i se calcula la proporción de ejecuciones (sobre las N repeticiones) en las que su traducción resultó sintácticamente válida tras el ciclo completo:

$$\text{Estabilidad}(i) = \frac{\text{ejecuciones en que el caso } i \text{ fue válido}}{N} \tag{5.3}$$

Un caso con estabilidad igual a 1 se traduce correctamente en todas las ejecuciones; un caso con estabilidad cercana a 0 resiste sistemáticamente la traducción. Esta métrica permite identificar patrones en los escenarios que resultan problemáticos para el LLM, como estructuras sintácticas inusuales o niveles de complejidad que exceden la capacidad del modelo. La estabilidad complementa la visión agregada de SVR con un análisis a nivel de caso individual, necesario para distinguir fallos aleatorios, que se resuelven en ejecuciones posteriores, de fallos sistemáticos, que persisten independientemente de la estocasticidad del modelo.

5.4.4. Métricas operativas

Se registra el tiempo de traducción por caso y el tiempo total del pipeline por ejecución. Si bien estas métricas no inciden directamente en la calidad de la traducción, permiten evaluar la viabilidad práctica del sistema y cuantificar la sobrecarga computacional introducida por el ciclo de corrección iterativo. En particular, la comparación del tiempo promedio por ronda permite estimar si el costo de cada iteración adicional se justifica por la ganancia en SVR que produce.

5.5. Resultados

Esta sección reporta los resultados finales obtenidos sobre el conjunto *test* de 90 escenarios, con $N = 5$ corridas por configuración y `max-rounds=3`, siguiendo el protocolo definido en la Sección 5.2. En consistencia con la Sección 3.1, en todos los casos se utilizó el proveedor/modelo `openai/gpt-4o`.

Las Tablas 5.2 y 5.3 presentan las métricas comparables entre configuraciones, reportadas como media y desvío estándar sobre las cinco corridas.

Tabla 5.2: Métricas de calidad por configuración en *test* (90 casos), $N = 5$

Config.	**SVR$_{\text{ini}}$** (%)	**SVR$_{\text{fin}}$** (%)	**Ganancia** (pp)	**Recovery** (%)
EN–EN	78.00 ± 4.40	89.78 ± 2.65	11.78 ± 3.57	53.03 ± 11.13
ES–EN	45.78 ± 2.41	86.00 ± 2.56	40.22 ± 4.87	73.99 ± 5.64
EN–ES	75.11 ± 3.90	91.11 ± 1.11	16.00 ± 3.65	63.67 ± 6.70
ES–ES	46.00 ± 2.02	84.22 ± 2.41	38.22 ± 4.20	70.64 ± 5.39

Tabla 5.3: Métricas operativas por configuración en *test* (90 casos), $N = 5$

Config.	**Rondas**	**Tiempo total (s)**
EN–EN	3.00	877.84 ± 129.38
ES–EN	3.00	1240.36 ± 52.52
EN–ES	3.00	1017.89 ± 168.57
ES–ES	3.00	1295.41 ± 117.56

Para facilitar la lectura de la incertidumbre con $N = 5$, la Tabla 5.4 resume el semiancho del intervalo de confianza al 95 % de las métricas de calidad.

Tabla 5.4: Semiancho del IC95 por configuración en *test* (90 casos), $N = 5$

Config.	**SVR$_{\text{ini}}$**	**SVR$_{\text{fin}}$**	**Ganancia**	**Recovery**
EN–EN	±5.47	±3.29	±4.43	±13.82
ES–EN	±2.99	±3.18	±6.04	±7.00
EN–ES	±4.84	±1.38	±4.53	±8.32
ES–ES	±2.51	±2.99	±5.22	±6.69

En términos de calidad final, las configuraciones con prompt en inglés (EN–EN y EN–ES) combinan valores altos de SVR$_{fin}$ con menor cantidad de fallos residuales por corrida. En particular, EN–ES alcanza el mayor promedio final (91.11 %) y también el menor volumen de instancias inválidas al cierre del ciclo (40 sobre 450 casos evaluados en total para esa configuración). En contraste, las configuraciones con prompt en español muestran mayor dependencia del ciclo de corrección: parten con SVR$_{ini}$ más bajo y cierran con mayor cantidad de fallos residuales (63 en ES–EN y 71 en ES–ES sobre 450).

Desde la perspectiva operativa, el tiempo total medio presenta diferencias consistentes entre configuraciones: EN–EN es la más rápida (877.84 s), seguida por EN–ES (1017.89 s), mientras que ES–EN y ES–ES exhiben tiempos superiores (1240.36 s y 1295.41 s, respectivamente). Dado que la media de

rondas utilizadas es 3.00 en los cuatro casos, esta diferencia temporal es coherente con la mayor carga de correcciones en las configuraciones que parten con menor validez inicial.

Al descomponer los tiempos de traducción por nivel de complejidad en el conjunto completo de corridas (20 ejecuciones, 1800 traducciones), se observa que los casos etiquetados como complejos son, en promedio, los más costosos de procesar (7.58 s por caso). Este resultado es consistente con la definición de complejidad adoptada en el dataset: escenarios con más entidades, interacciones y condiciones requieren mayor elaboración por parte del modelo para producir la salida IBDD.

5.5.1. Casos no convergentes tras tres rondas

El análisis de los archivos de validación finales (`*_validation.json`) permite identificar los casos que permanecen inválidos al agotar `max-rounds=3`. Sobre 20 ejecuciones totales (4 configuraciones $\times$ 5 corridas), se registran 220 instancias inválidas finales. No obstante, la no convergencia persistente es acotada: solo dos casos fallan en las cinco corridas de una misma configuración, el caso 68 en ES–EN y el caso 47 en ES–ES.

En estos casos persistentes, los mensajes de error se concentran en patrones sintácticos recurrentes: uso no esperado de corchetes en expresiones (`No terminal matches '['`), separación inválida de argumentos/listas (`No terminal matches ','`) y expresiones incompletas al final de la entrada (`Unexpected end-of-input`). En el conjunto completo de errores residuales, estos patrones aparecen con alta frecuencia (48, 33 y 34 ocurrencias, respectivamente), junto con fallos de sintaxis de gates (`?/!`, 12 ocurrencias).

Estos resultados muestran que, aunque el ciclo iterativo corrige una fracción importante de fallos iniciales, persiste un subconjunto de escenarios con estructuras que el sistema no estabiliza dentro de tres rondas. Esta evidencia delimita el principal frente de mejora del enfoque actual: robustecer la generación y/o las reglas de corrección para construcciones con listas, indexación y expresiones compuestas en guardas.

5.6. Análisis

El análisis se realiza exclusivamente sobre métricas agregadas de validación y error obtenidas en *test* (90 casos).

5.6.1. RQ1: validez sintáctica de las traducciones

En las cuatro configuraciones, la validez sintáctica final media supera el 84 %: EN–ES alcanza 91.11 %, EN–EN 89.78 %, ES–EN 86.00 % y ES–ES 84.22 %. Bajo el protocolo definido ($N = 5$, `max-rounds=3`), estos valores ubican la conformidad sintáctica final del sistema en el rango 84.22–91.11 % sobre el conjunto *test*.

La dispersión de SVR_{fin} es baja (DE entre 1.11 y 2.65 puntos), lo que sugiere un comportamiento relativamente estable entre corridas para la métrica principal. En términos de robustez, las configuraciones con prompt en inglés (EN–EN y EN–ES) muestran mejores resultados finales y baja variabilidad entre corridas.

5.6.2. RQ2: efecto del idioma de prompt y dataset

Los resultados muestran una diferencia clara en SVR_{ini} según el idioma del prompt: las configuraciones con prompt en inglés (EN–EN, EN–ES) parten de niveles iniciales altos (78.00 % y 75.11 %), mientras que las configuraciones con prompt en español (ES–EN, ES–ES) parten de niveles iniciales significativamente menores (45.78 % y 46.00 %).

Tras el ciclo de corrección, la brecha se reduce en SVR_{fin}: las cuatro configuraciones quedan en el rango 84.22–91.11 %. Por lo tanto, en este experimento el idioma del prompt se asocia fuertemente con la calidad inicial, pero su efecto sobre la calidad final es menor una vez aplicado el mecanismo iterativo. El idioma del dataset también incide, aunque con efecto más moderado que el observado para el prompt en la etapa inicial.

5.6.3. RQ3: efectividad del ciclo de corrección iterativo

La ganancia media en puntos porcentuales confirma que el ciclo de corrección aporta mejoras en todas las configuraciones: +11.78 (EN–EN), +40.22 (ES–EN), +16.00 (EN–ES) y +38.22 (ES–ES). Asimismo, la tasa de recuperación de casos inicialmente fallidos es positiva en todas las configuraciones, con valores entre 53.03 % y 73.99 %.

No obstante, la métrica `rounds_used.mean` es 3.00 en todas las configuraciones, lo que indica que en promedio se utiliza el máximo de rondas permitido. Esto sugiere que el ciclo mejora de forma sustantiva la validez, pero no alcanza convergencia temprana bajo el límite actual de iteraciones.

5.6.4. Interpretación de resultados

Los resultados sobre el conjunto *test* (90 casos) muestran que el enfoque híbrido propuesto es efectivo para producir traducciones IBDD sintácticamente válidas en las cuatro configuraciones experimentales. En términos agregados, la validez final se ubicó entre 84.22 % y 91.11 % (Tabla 5.2), lo que indica que la combinación de generación con LLM y validación formal permite sostener un nivel alto de conformidad con la gramática objetivo.

Un hallazgo central es la diferencia entre desempeño inicial y final. Las configuraciones con prompt en español parten de una validez inicial sustancialmente menor, pero recuperan gran parte de esa brecha tras aplicar corrección iterativa. En consecuencia, el mecanismo de reintento no actúa como un ajuste marginal, sino como un componente estructural del pipeline para estabilizar la calidad final entre configuraciones con condiciones lingüísticas distintas.

La interpretación de RQ2 sugiere que el idioma del prompt está más asociado con la calidad inicial que el idioma del dataset. No obstante, dado el tamaño muestral por configuración ($N = 5$), esta evidencia debe leerse como patrón empírico consistente dentro del protocolo adoptado y no como una afirmación causal general sobre cualquier modelo, prompt o dominio.

Desde la perspectiva operativa, todas las configuraciones consumen en promedio el máximo de rondas permitido (`max-rounds=3`), lo que indica que el sistema mejora de manera sistemática, pero no converge tempranamente en el régimen actual. Este comportamiento es coherente con la existencia de un subconjunto de casos persistentemente difíciles, donde el diagnóstico y reintento corrigen parte de los errores, aunque no eliminan completamente la falla sintáctica residual.

En relación con trabajos previos, los resultados acompañan la literatura que reporta mejor desempeño de modelos GPT en tareas de generación estructurada para contextos BDD, especialmente cuando se utiliza prompting con ejemplos. Al mismo tiempo, este trabajo extiende ese marco al incorporar validación formal determinística y corrección iterativa en un pipeline end-to-end BDD→IBDD, aspecto menos desarrollado en estudios centrados únicamente en calidad textual de salida.

5.6.5. Limitaciones

La principal limitación del estudio es que la evaluación cuantitativa se concentra en validez sintáctica. Una traducción aceptada por la gramática IBDD puede contener desalineaciones semánticas respecto del escenario BDD original, por lo que la métrica SVR no agota la noción de calidad de traducción.

Una segunda limitación es el tamaño del muestreo estocástico por configu-

ración ($N = 5$). Este diseño permite estimar media, dispersión e intervalos de confianza, pero restringe el poder para comparaciones inferenciales más finas entre configuraciones cercanas.

También debe considerarse la dependencia del prompt y del conjunto de reglas de traducción utilizados. Parte del desempeño observado refleja decisiones de diseño del prompt y del mecanismo de corrección, por lo que cambios en estas piezas pueden modificar los resultados sin alterar la arquitectura general.

Finalmente, la evaluación se realizó con un único modelo seleccionado y un conjunto acotado de dominios dentro del dataset construido. Aunque esto es adecuado para la pregunta de investigación de este trabajo, no garantiza transferencia automática a otros modelos, proveedores o colecciones de escenarios con distribuciones lingüísticas distintas.

5.6.6. Consideraciones de validez

Para la validez interna, se mitigaron riesgos mediante partición fija dev/-test, repetición de corridas por configuración y uso de un validador sintáctico determinístico. Aun así, persisten posibles fuentes de variación asociadas a la naturaleza estocástica del modelo y a cambios no controlados en el entorno del proveedor.

Para la validez de constructo, la métrica principal (SVR) representa de forma directa la conformidad gramatical, pero no captura fidelidad semántica. Por lo tanto, las conclusiones deben entenderse estrictamente en términos de corrección sintáctica de la traducción.

Para la validez externa, los hallazgos se sustentan en un dataset de 120 escenarios con evaluación final sobre 90 casos de test y cuatro configuraciones lingüísticas. Este alcance ofrece evidencia relevante para el contexto estudiado, pero su generalización a otros dominios, tamaños de dataset o configuraciones de prompting requeriría validación adicional.

Capítulo 6

Conclusiones y Trabajo Futuro

6.1. Conclusiones

Este trabajo abordó la automatización de la traducción de escenarios BDD en lenguaje natural hacia representaciones IBDD, una etapa crítica del pipeline BDD-a-MBT que, en la práctica, suele requerir intervención manual experta. Para ello se diseñó e implementó un enfoque híbrido que combina un módulo de traducción basado en LLM con validación sintáctica determinística mediante gramática formal IBDD y parser Earley, complementado por un ciclo iterativo de corrección sobre casos fallidos.

Desde el punto de vista de la ingeniería, el trabajo aporta: (i) un pipeline reproducible, con proveedor/modelo, prompt y dataset parametrizados; (ii) una gramática operacional de IBDD integrada a un validador automático; (iii) un mecanismo de corrección asistido por diagnóstico de errores sintácticos; y (iv) una implementación organizada en etapas desacopladas, con resultados intermedios persistidos en archivos JSON, lo que favorece trazabilidad, reproducibilidad y ejecución independiente de cada componente.

En términos empíricos, la evaluación final sobre el conjunto *test* de 90 escenarios mostró que el sistema alcanza niveles altos de validez sintáctica final en las cuatro configuraciones analizadas, con medias entre 84.22 % y 91.11 %. Además, la variabilidad entre corridas se mantuvo acotada para la métrica principal, lo que respalda la estabilidad del comportamiento observado bajo el protocolo experimental adoptado.

Respecto de las preguntas de investigación:

RQ1: El sistema logra traducir automáticamente escenarios BDD a IBDD con alta tasa de conformidad sintáctica en el conjunto *test*, medida de forma objetiva mediante el parser.

RQ2: El idioma del prompt muestra una asociación más marcada con la validez inicial que el idioma del dataset; luego del ciclo de corrección, las diferencias entre configuraciones se reducen.

RQ3: El ciclo iterativo de corrección aporta mejoras consistentes en todas las configuraciones (ganancias positivas de SVR y recuperación de casos inicialmente fallidos), aunque en promedio utiliza el máximo de rondas configurado.

En síntesis, los resultados sostienen que la combinación de generación con LLM y validación/corrección formal constituye una estrategia viable para automatizar la etapa BDD→IBDD en términos de validez sintáctica, dentro del contexto experimental evaluado. Al mismo tiempo, la evidencia obtenida delimita con claridad el alcance actual del sistema: bajo el protocolo evaluado se observa alta validez sintáctica, mientras que la fidelidad semántica de la traducción permanece como dimensión abierta.

6.2. Trabajo Futuro

La primera línea de evolución propuesta es explorar una extensión del pipeline actual hacia un esquema con agentes especializados. En problemas donde la resolución requiere múltiples decisiones interdependientes, un flujo estrictamente lineal ofrece menor capacidad para representar estrategias de búsqueda, verificación cruzada y corrección iterativa. En este marco, agentes especializados podrían asumir funciones diferenciadas (traducción, validación, diagnóstico, propuesta de corrección y control de calidad), coordinados por una política de orquestación explícita. Como extensión concreta, esta arquitectura debería formalizarse y documentarse de manera sistemática, especificando roles, contratos de entrada y salida, criterios de activación entre agentes y registro de decisiones, con el objetivo de preservar reproducibilidad y auditabilidad. Bajo este esquema, el sistema podría incorporar ciclos de verificación cruzada entre agentes, mecanismos de consenso para resolver conflictos y estrategias de reintento guiadas por diagnósticos intermedios.

El sistema desarrollado en este trabajo implementa validación sintáctica de las traducciones IBDD mediante una gramática formal y un parser, pero no incorpora una etapa de validación semántica automática que verifique si la traducción preserva correctamente el significado del escenario BDD original. Esto constituye una limitación de alcance: una traducción puede ser sintácticamente correcta y al mismo tiempo semánticamente deficiente; por ejemplo, podría omitir una entidad mencionada en el escenario, utilizar un

gate de salida donde el BDD describe una acción del usuario, o perder un valor numérico relevante para una postcondición.

Durante el desarrollo de este trabajo se consideraron alternativas preliminares para abordar parcialmente este problema, por ejemplo mediante heurísticas léxicas (cotejo de entidades, acciones y valores numéricos entre el texto BDD y la traducción IBDD), verificaciones de consistencia numérica, uso del LLM como apoyo para revisión semántica, o validadores apoyados en reglas de dominio. Sin embargo, no se avanzó con una implementación de estas estrategias, ya que su diseño y evaluación exceden el alcance de este trabajo. En particular, la naturaleza abreviada de IBDD, donde variables como `JF` representan entidades como "job file" y gates como `?submit` condensan frases completas, dificulta el cotejo directo con el lenguaje natural. Por ello, la validación semántica permanece como una línea de trabajo futuro abierta.

Otra dirección de trabajo futuro es la generalización de la capa de integración con proveedores LLM. Si bien la arquitectura actual opera con proveedor y modelo parametrizados por configuración y desacopla el resto del pipeline del cliente concreto, la implementación disponible soporta explícitamente dos proveedores: OpenAI (utilizado en las corridas principales tras la selección del modelo en el Capítulo 3.1) y Ollama (incluido para habilitar ejecución local de modelos de código abierto en contextos donde se prioriza soberanía de datos). Extender el sistema a otros proveedores (por ejemplo, Anthropic) requiere implementar un adaptador adicional en la clase `LLMClient`, en particular para resolver diferencias en autenticación, formato de mensajes, manejo de errores y soporte de salidas estructuradas. Una evolución natural del sistema consiste en formalizar esta capa como un conjunto de adaptadores intercambiables con capacidades declaradas, de modo de aproximar una interoperabilidad más completa entre proveedores sin modificar los módulos de traducción, análisis de errores y orquestación.

Una línea de trabajo futuro con impacto directo es la evaluación del pipeline completo BDD→IBDD→BDDTS→STS→Tests. En este trabajo, la evaluación se centró en la validez sintáctica de la traducción a IBDD; como extensión, resulta relevante medir cómo esos resultados se propagan a las etapas formales posteriores y a la calidad de los casos de prueba generados. Esto permitiría incorporar métricas de extremo a extremo, por ejemplo proporción de escenarios que llegan a tests ejecutables, cobertura alcanzada por los tests sobre el modelo STS y tasa de fallas atribuibles a errores introducidos en la etapa de traducción. Esta evaluación integral ofrecería evidencia más fuerte sobre la utilidad práctica del sistema en un flujo MBT completo.

Apéndice A

Prompt de Traducción Utilizado

Este apéndice reproduce el prompt en español utilizado en los experimentos de traducción BDD→IBDD. La versión fuente empleada por el sistema puede encontrarse en el archivo `docs/PROMPT_ES.md` del repositorio. La versión correspondiente en inglés puede encontrarse en `docs/PROMPT_EN.md`.

A.1. Contexto

Vas a traducir escenarios escritos en formato Gherkin a un lenguaje intermedio llamado IBDD (*Intermediate Behavior-Driven Development*). IBDD mantiene la estructura Given–When–Then pero añade precisión formal mediante sistemas de transición simbólicos.

A.2. Definición de IBDD

A.2.1. Gramática

```
Scenario ::= Given When Then
Given ::= 'GIVEN' Declaration '[' Guard ']'
Declaration ::= lv1, ..., lvn
Guard ::= P | Guard '∧' Guard
When ::= 'WHEN' Switch+
Switch ::= Interaction Condition Assignment
Interaction ::= g. iv1, ..., ivn
Condition ::= B
Assignment ::= A
Then ::= 'THEN' Switch* '[' Guard ']'
```

A.2.2. Elementos Clave

- Variables locales (`lv`): se declaran en `GIVEN` y son visibles en todo el escenario.
- Variables globales: representan estado persistente del sistema, por ejemplo `SJL` (*Scheduled Jobs List*).
- Variables de interacción (`iv`): modelan la comunicación entre el sistema y su entorno.
- Gates: el prefijo `!` denota salidas del sistema y el prefijo `?` denota entradas desde el entorno.
- Guards: expresan condiciones booleanas.
- Assignments: expresan asignaciones del tipo `variable := expresión`.

A.3. Ejemplos de Traducción

A.3.1. Ejemplo 1: escenario simple con variable local

Escenario Gherkin

```
{
"id": 1,
"domain": "printer",
"title": "Submit job to printer",
"given": "a job file",
"when": "the operator submits the job file
using Submission method",
"then": "the printer appends a new controller job
to the scheduled jobs AND the controller job
is of type Job type"
}
```

Traducción IBDD

```
GIVEN JF [true]
WHEN ?submit.jf,sm
true
JF := jf
THEN !append.cj,sjl
cj.type = JT ∧ sjl = SJL ∧ JT = getJobType(SM)
CJ := cj, SJL := add(CJ, SJL)
[is_in_list(CJ, SJL) ∧ CJ.type = JT]
```

Razonamiento

- La cláusula Given declara la variable local `JF` sin precondiciones, por lo que se utiliza la guarda `[true]`.
- La cláusula When se modela mediante el gate de entrada `?submit` y las variables de interacción `jf` y `sm`.
- La cláusula Then utiliza el gate de salida `!append` y una postcondición que verifica la inclusión del job en la lista.

A.3.2. Ejemplo 2: variables locales con guardas y acciones secuenciales

Escenario Gherkin

```
{
"id": 2,
"domain": "printer",
"title": "Move production job to printed jobs",
"given": "a controller job is in the scheduled jobs
AND the controller job is a production job",
"when": "the printer starts printing the controller job
AND the printer completes printing the controller job",
"then": "the controller job is in the printed jobs"
}
```

Traducción IBDD

```
GIVEN CJ [is_in_list(CJ, SJL) ∧ CJ.type = Production]
WHEN !printstart.cj
cj.id = CJ.id ∧ cj.state = Printing
CJ.state := cj.state
!printcomplete.cj
cj.id = CJ.id ∧ cj.state = Completed
CJ.state := cj.state, PJL := add(CJ, PJL)
THEN [is_in_list(CJ, PJL)]
```

Razonamiento

- La cláusula Given declara la variable local `CJ` y expresa una guarda compuesta sobre su pertenencia a `SJL` y su tipo.

- La cláusula When se traduce como dos switches secuenciales, uno para el inicio de impresión y otro para su finalización.
- La cláusula Then verifica que el job haya pasado a la lista de impresiones completadas.

A.3.3. Ejemplo 3: escenario con guarda sin variables locales

Escenario Gherkin

```
{
"id": 3,
"domain": "printer",
"title": "Clean up printed jobs on restart",
"given": "the printer controller is running",
"when": "the operator restarts the printer controller",
"then": "the printed jobs clean up is executed"
}
```

Traducción IBDD

```
GIVEN [is_running(CTRL)]
WHEN ?restart
true
(no assignment)
THEN !cleanup
true
PJL := empty(PJL)
[is_empty(PJL)]
```

Razonamiento

- La cláusula Given no introduce variables locales; solo impone una guarda sobre el estado del controlador.
- La cláusula When se modela mediante un gate de entrada simple.
- La cláusula Then representa una acción del sistema seguida de una postcondición que verifica el vaciado de la lista.

A.3.4. Ejemplo 4: múltiples variables de interacción

Escenario Gherkin

```
{
"id": 4,
"domain": "ecommerce",
"title": "Add product to cart",
"given": "a product with price and a shopping cart",
"when": "the user adds the product to the cart
with quantity",
"then": "the cart contains the product
AND the total price is updated"
}
```

Traducción IBDD

```
GIVEN P, SC [P.price >0 ∧ is_valid(SC)]
WHEN ?add_to_cart.p,sc,qty
p.id = P.id ∧ sc.id = SC.id ∧ qty >0
SC.items := add(p, SC.items),
SC.total := SC.total + (p.price * qty)
THEN [is_in_list(P, SC.items) ∧
SC.total = calculate_total(SC.items)]
```

Razonamiento

- La cláusula Given introduce dos variables locales y una guarda compuesta.
- La cláusula When utiliza múltiples variables de interacción: producto, carrito y cantidad.
- La cláusula Then verifica tanto la inclusión del producto como la actualización correcta del total.

A.4. Formato de Entrada y Salida

La entrada del sistema consiste en un arreglo JSON de escenarios, donde cada objeto contiene los campos `id`, `domain`, `title`, `given`, `when` y `then`. La salida debe mantener exactamente esa estructura y añadir el campo `ibdd_representation` con la traducción IBDD correspondiente.

A.5. Reglas de Traducción

A.5.1. Cláusula Given

- Identificar si el texto menciona variables locales, por ejemplo "a job" o "a product".
- Si la cláusula describe únicamente el estado del sistema, modelarla solo como guarda, sin declaración.
- Combinar múltiples condiciones mediante el operador $\wedge$.

A.5.2. Cláusula When

- Las acciones del usuario se traducen como gates de entrada.
- Las acciones del sistema se traducen como gates de salida.
- Las acciones múltiples conectadas por "AND" se traducen como switches secuenciales.
- Deben identificarse las variables de interacción necesarias para la comunicación.

A.5.3. Cláusula Then

- El estado final se expresa en la postcondición.
- Si hay acciones explícitas del sistema, deben modelarse como switches con gates de salida.
- La postcondición final siempre se escribe entre corchetes.

A.5.4. Convenciones de Nombres

- Variables locales: abreviaturas en mayúsculas, por ejemplo `CJ`, `JF`, `P`, `SC`.
- Variables globales: nombres descriptivos en mayúsculas, por ejemplo `SJL`, `PJL`, `CTRL`.
- Variables de interacción: nombres en minúscula relacionados con las variables locales, por ejemplo `cj`, `jf`, `p`, `sc`.
- Gates: verbos descriptivos en minúscula.
- Ejemplos representativos son `submit`, `printstart` y `add_to_cart`.

A.6. Instrucción Final

Procesar el JSON de entrada y devolver el mismo JSON con el campo `ibdd_representation` añadido a cada escenario. No deben incluirse explicaciones, comentarios ni marcas de formato adicionales; solo debe devolverse el JSON resultante, manteniendo exactamente la estructura de la entrada. La representación IBDD debe serializarse como una cadena con saltos de línea.

Bibliografía

[1] Petra van den Bos y Jan Tretmans. "Coverage-Based Testing with Symbolic Transition Systems". En: *Tests and Proofs (TAP 2019)*. Lecture Notes in Computer Science. Springer, 2019.

[2] Laura Brandán Briones. "Theories for Model-based Testing: Real-time and Coverage". Tesis doct. Enschede, The Netherlands: University of Twente, 2007.

[3] Cucumber Ltd. *Cucumber Open*. `https://cucumber.io/`. 2024.

[4] Cucumber Ltd. *Gherkin Reference*. `https://cucumber.io/docs/gherkin/reference/`. 2024.

[5] Dailymotion Engineering. *Tartiflette: a Python 3.6+ GraphQL engine built with Lark*. 2019. URL: `https://github.com/tartiflette/tartiflette` (visitado 28-01-2026).

[6] Jay Earley. "An Efficient Context-Free Parsing Algorithm". En: *Communications of the ACM* (1970).

[7] Steven Y. Feng et al. "A Survey of Data Augmentation Approaches for NLP". En: *Findings of the Association for Computational Linguistics (ACL-IJCNLP)*. Association for Computational Linguistics, 2021.

[8] Jannik Fischbach et al. "SPECMATE: Automated Creation of Test Cases from Acceptance Criteria". En: *2020 IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2020.

[9] Lars Frantzen, Jan Tretmans y Tim A. C. Willemse. "A Symbolic Framework for Model-Based Testing". En: *Formal Approaches to Software Testing and Runtime Verification (FATES/RV 2006)*. Vol. 4262. Lecture Notes in Computer Science. Springer, 2006.

[10] Lars Frantzen, Jan Tretmans y Tim A. C. Willemse. "Test Generation Based on Symbolic Specifications". En: *Formal Approaches to Software Testing (FATES 2004)*. Lecture Notes in Computer Science. Springer, 2005.

[11] Dick Grune y Ceriel J. H. Jacobs. *Parsing Techniques: A Practical Guide*. 2nd. Monographs in Computer Science. Springer, 2008.

[12] Ari Holtzman et al. "The Curious Case of Neural Text Degeneration". En: *International Conference on Learning Representations (ICLR)*. 2020.

[13] Eirik Nøstdahl Hundvin. "Code Fragment Generation from Lark Grammar Specifications". Tesis de mtría. Bergen, Norway: University of Bergen, 2022.

[14] Juyong Jiang et al. "A Survey on Large Language Models for Code Generation". En: *ACM Transactions on Software Engineering and Methodology* (2024).

[15] Shanthi Karpurapu et al. "Comprehensive Evaluation and Insights into the Use of Large Language Models in the Automation of Behavior-Driven Development Acceptance Test Formulation". En: *IEEE Access* (2024). DOI: 10.1109/ACCESS.2024.3391815. URL: https://ieeexplore.ieee.org/document/10506519.

[16] Lark Parser Project. *Lark Documentation*. 2026. URL: https://lark-parser.readthedocs.io/en/stable/ (visitado 29-01-2026).

[17] Lark Parser Project. *Lark Issue Tracker*. 2026. URL: https://github.com/lark-parser/lark/issues (visitado 29-01-2026).

[18] Lark Parser Project. *Lark: a parsing toolkit for Python*. 2026. URL: https://github.com/lark-parser/lark (visitado 28-01-2026).

[19] Robert M. Metcalfe y David R. Boggs. "Ethernet: Distributed Packet Switching for Local Computer Networks". En: *Communications of the ACM* (1976).

[20] Dan North. *Introducing BDD*. https://dannorth.net/blog/introducing-bdd/. Originally published in Better Software magazine, March 2006. 2006.

[21] OpenAI. *GPT-4 Technical Report*. 2023. arXiv: 2303.08774 [cs.CL].

[22] OpenAI. *Introducing Structured Outputs in the API*. https://openai.com/index/introducing-structured-outputs-in-the-api/. Accessed: 2025. 2024.

[23] Python Packaging Authority. *Lark on PyPI*. 2026. URL: https://pypi.org/project/lark/ (visitado 29-01-2026).

[24] Python Poetry Team. *poetry-core*. 2026. URL: https://github.com/python-poetry/poetry-core (visitado 28-01-2026).

[25] python-parsing-benchmarks contributors. *python-parsing-benchmarks*. 2026. URL: https://github.com/goodmami/python-parsing-benchmarks (visitado 29-01-2026).

[26] python-parsing-benchmarks contributors. *python-parsing-benchmarks*. 2026. URL: https://github.com/goodmami/python-parsing-benchmarks (visitado 28-01-2026).

[27] Matthew Renze y Erhan Guven. "The Effect of Sampling Temperature on Problem Solving in Large Language Models". En: *Findings of the Association for Computational Linguistics: EMNLP 2024* (2024).

[28] Rico Sennrich, Barry Haddow y Alexandra Birch. "Neural Machine Translation of Rare Words with Subword Units". En: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*. Berlin, Germany: Association for Computational Linguistics, 2016, págs. 1715-1725.

[29] Mathias Soeken, Robert Wille y Rolf Drechsler. "Assisted Behavior Driven Development Using Natural Language Processing". En: *Objects, Models, Components, Patterns (TOOLS 2012)*.

[30] Carlos Solís y Xiaofeng Wang. "A Study of the Characteristics of Behaviour Driven Development". En: *2011 37th EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA)*. Oulu, Finland: IEEE, 2011.

[31] Jan Tretmans. "Model Based Testing with Labelled Transition Systems". En: *Formal Methods and Testing*. Lecture Notes in Computer Science. Springer, 2008.

[32] Mark Utting, Alexander Pretschner y Bruno Legeard. "A Taxonomy of Model-Based Testing Approaches". En: *Software Testing, Verification and Reliability* 22.5 (2012).

[33] Ashish Vaswani et al. "Attention Is All You Need". En: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017.

[34] Vyper Team. *Vyper: Pythonic smart contract language for the EVM*. 2026. URL: https://github.com/vyperlang/vyper (visitado 28-01-2026).

[35] Niklaus Wirth. "What Can We Do about the Unnecessary Diversity of Notation for Syntactic Definitions?" En: *Communications of the ACM* (1977).

[36] Tannaz Zameni et al. “An Intermediate Language to Integrate Behavior-Driven Development Scenarios and Model-Based Testing”. En: *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering - Companion (SANER-C)*. IEEE, 2024.

[37] Tannaz Zameni et al. “From BDD Scenarios to Test Case Generation”. En: *2023 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 2023.

[38] Tannaz Zameni et al. “Sequential Composition of BDD Transition Systems for Model-Based Testing”. En: *Formal Techniques for Distributed Objects, Components, and Systems (FORTE)*. Lecture Notes in Computer Science. Springer, 2025.